Learning with AI

A Student and Teacher Guide

Building Genuine Understanding
When the Answers Are Just a Prompt Away

J.M. Hyman

2026

Copyright © 2026 J.M. Hyman.
Released under the Creative Commons Attribution 4.0 International License (CC-BY-4.0). You are free to share and adapt this work, including for commercial purposes, provided you give appropriate credit.

Citation: Hyman, J.M. (2026). *Learning with AI: A Student and Teacher Guide.* https://github.com/machyman/learning-with-ai-student-guide

Version v2026.2.

Contents

Preface vii

What this guide is and isn't ix

A note on examples xi

I For Students 1

1 Two Ways to Use AI 3
1.1 Why the distinction matters . . . 3
1.2 A concrete contrast . . . 3
1.3 The maxim . . . 4

2 Six Things AI Does Well 5
2.1 Clarifying notation and definitions . . . 5
2.2 Generating alternative explanations . . . 6
2.3 Suggesting counterexamples and edge cases . . . 6
2.4 Computational prototyping . . . 7
2.5 Identifying what to study . . . 8
2.6 Self-testing through retrieval practice . . . 8
2.7 What the six have in common . . . 9

3 Six Things AI Does Badly 11
3.1 Citation hallucination . . . 11
3.2 Unverified analysis acceptance . . . 11
3.3 Meaning drift in technical prose . . . 12
3.4 Code that runs but does the wrong thing . . . 13
3.5 Source conflation . . . 13
3.6 Multi-language divergence . . . 14
3.7 The shape of these failures . . . 14

4 The Verification Principle 15
4.1 Dimensional verification . . . 15
4.2 Special-case reduction . . . 16
4.3 Numerical spot-check . . . 16
4.4 A generic verification checklist . . . 17
4.5 Why verification feels expensive but isn't . . . 18

5 Cognitive Offloading and the Evidence 19
5.1 Retrieval practice: why effortful recall outperforms re-reading . . . 19
5.2 Productive failure: why struggling first beats being told . . . 20
5.3 The illusion of fluency . . . 21
5.4 Cognitive offloading: when outsourcing skills permanently weakens them . . . 21
5.5 What this means in practice: specific habits . . . 22

6 Disclosure and Intellectual Honesty **23**
6.1 Why disclosure matters . . . 23
6.2 The five-field disclosure template . . . 23
6.3 Three worked examples . . . 24
6.4 When disclosure matters and when it doesn't . . . 25
6.5 Disclosure as a habit, not a chore . . . 26

7 Working with AI on Different Tasks **27**
7.1 Studying for an exam . . . 27
7.2 Working a problem set . . . 28
7.3 Reading a difficult paper . . . 28
7.4 Writing a paper or report . . . 29
7.5 Conducting original research . . . 30

8 Self-Assessment and Common Traps **31**
8.1 Four self-tests . . . 31
8.2 Four common traps . . . 32
8.3 Closing argument: the long horizon . . . 33

II For Instructors **35**

9 Teaching with AI: The Pedagogical Stance **37**
9.1 What the course is for . . . 37
9.2 Five principles that organize the rest of Part II . . . 37
9.3 What AI does not change about the instructor's role . . . 38
9.4 What AI does change . . . 38

10 Course Design: Three Tool Regimes **41**
10.1 No-AI assessments . . . 41
10.2 Bounded-AI assessments . . . 42
10.3 Open-AI assessments . . . 43
10.4 Recommended weighting . . . 43

11 Assessment Instruments **45**
11.1 Eight competency domains . . . 45
11.2 Four assessment instrument types . . . 46
11.3 Grading rubrics . . . 47
11.4 Calibration and consistency . . . 47

12 Documentation, Disclosure, and Academic Integrity **49**
12.1 What students must document . . . 49
12.2 Minimal integrity boundaries . . . 49
12.3 The integrity-under-questioning standard . . . 50
12.4 What to do when integrity fails . . . 50
12.5 The instructor's own disclosure . . . 51

A Generic Verification Checklist **53**

B Pocket Disclosure Template **55**

C Further Reading **57**

D Sample Course Syllabus Statements **59**

E Oral Exam Question Bank **61**

E.1 Definitions and motivating examples 61
E.2 Short proofs 61
E.3 Hypothesis stress tests 62
E.4 Audit-and-repair 62
E.5 Synthesis questions 63
E.6 Question selection and pacing 63

Preface

You have access to AI tools that can solve most homework problems, write most essays, debug most code, and explain most textbook chapters in three different ways before you finish reading the original. Your professors know this. Your peers are using these tools, and you probably are too. The question is not whether you will use AI in your studies. The question is whether the way you use it will help you learn or quietly prevent you from learning.

This guide takes a position. The position is that AI tools, used carelessly, are the most efficient mechanism ever invented for producing the appearance of understanding without the substance. Used carefully, the same tools are among the most powerful study aids in the history of education. The difference between the two outcomes is not the AI; it is the habits you build around it.

I wrote this guide because the students I see in my own courses and the readers of my textbooks ask the same questions. *When should I use AI to help me with a problem? When does AI use cross from learning aid into shortcut? How do I tell whether what the AI said is actually true? How do I disclose AI use without sounding like I'm confessing to something?* The questions are good. The answers are not yet widely known. This guide collects what I have found useful, grounds it in what cognitive science says about how people actually learn, and provides concrete habits you can adopt this week.

The guide is short by design. You can read it in an evening. The chapters are independent enough that you can dip into the ones relevant to a current problem, but the order matters: Chapter 1 sets up a distinction the rest of the guide depends on, and Chapter 4 introduces a verification habit that the later chapters assume. Read those two even if you skip the rest.

The examples are drawn from mathematics, physics, statistics, programming, and the sciences. If you study in one of these fields, you will see your discipline. If you study in another quantitative or technical field, you will see patterns that transfer. The guide is not subject-specific.

I am writing this in 2026. AI tools will change. The specific examples in the guide may date. The habits the guide teaches—verification, disclosure, retrieval practice, productive struggle—will not.

A note on the structure. This guide is divided into two parts. **Part I** is for students. It develops the habits and principles that distinguish productive AI use from passive AI use, with worked examples drawn from across the technical disciplines. **Part II** is for instructors. It addresses the harder problem of how to design a course that teaches students to use AI well: which assessments work, how to weight competencies, how to structure oral examinations, what to require of students by way of disclosure. The two parts share the same pedagogical foundation, and a thorough reading of Part I is the right preparation for Part II.

A teacher who is comfortable in their subject but new to AI-aware teaching will find Part I useful as preparation, and Part II as a working framework. A student who reads only Part I will get the full habit set the guide is designed to convey; a student curious about the rationale behind their instructor's policies may find Part II illuminating.

J.M. Hyman

2026

What this guide is and isn't

This guide is:

- A short, opinionated handbook on AI use in technical study—written for students in Part I, and for instructors in Part II.
- Concrete and practical, with worked examples on every conceptual claim.
- Grounded in cognitive-science evidence where evidence exists.
- Tool-agnostic and intended to age well: no product names, no version-specific behaviors.
- Reusable across disciplines and books. If a course uses a textbook that references this guide, the guide is the same one referenced by other courses students take.

This guide is not:

- A tutorial on any specific AI tool. The tool you use next semester may not be the tool you use today; the principles will outlast both.
- A technical introduction to how language models work. There are excellent resources on this; the guide assumes you can use AI tools competently and addresses how to use them in study and in teaching, not how they function.
- A statement of academic-integrity policy. Your institution sets policy; check what your institution requires. The guide addresses honesty practices and assessment designs that are good ideas in any policy environment.
- A neutral survey. The guide takes positions, and the positions are sometimes inconvenient. If you want a tour of the AI-in-education debate, this is not it.

A note on examples

The examples throughout this guide come from several disciplines. A reader studying calculus will see calculus examples; a reader writing code will see programming examples; a reader running statistical analyses will see statistical examples. I do not assume you study every discipline I draw from; I do assume that a habit illustrated in an unfamiliar example transfers cleanly to your own field once you see it. Where an example uses notation or vocabulary specific to one discipline, I introduce the notation in plain English at first use.

The first edition draws most heavily from mathematics, physics and engineering, statistics, and programming. Examples from the life and social sciences appear less frequently; examples from the humanities are sparse. This reflects my own training and the disciplinary contexts in which I have most often watched AI-aware practice succeed and fail. Subsequent editions will broaden the mix as I receive feedback from readers in fields the v1 examples underserve. The patterns the guide describes are general; only the example coverage is uneven.

Some readers will recognize themselves in the examples drawn from epidemiological modeling. That is not an accident—I am an applied mathematician who has spent a career on this subject, and the kinds of failures I see most clearly are the kinds I see in my own field. I have tried to balance the disciplinary mix so the guide does not read as a quietly subject-specific document. Readers who tell me a section reads as too narrowly drawn from one field will be doing me and future readers a kindness; please send corrections to the repository's issue tracker.

Part I

For Students

Chapter 1

Two Ways to Use AI

There are two ways to use an AI tool while studying. They look similar from the outside. They produce very different results.

A student facing a difficult problem opens an AI chat window. They type the problem, possibly in their own words, possibly copy-pasted from a textbook. The AI returns an answer. The student reads the answer, decides it makes sense, and moves on.

This is one way to use AI. It is fast. It produces an artifact. The student can submit the answer or paste it into a notebook and feel they have made progress. They have not learned much.

A different student, facing the same problem, opens the same AI chat window and types something else. They might ask: *"Explain three different methods for approaching this kind of problem and tell me which one is most appropriate here and why."* Or: *"What concept do I need to understand before I can solve this?"* Or: *"Pose three questions I should be able to answer before I attempt this problem."* The AI returns explanations, conceptual context, or a self-test. The student works through the material the AI has surfaced. Then the student attempts the problem.

This is the other way to use AI. It is slower. It does not immediately produce an answer to the problem. It produces something else: a scaffold for the student's own reasoning. After working through the scaffold, the student will solve the problem with a substantially better grasp of why the solution works than the first student ever achieves.

The first student is using AI to get answers. The second student is using AI to learn. The guide is written for students who want to be the second student.

1.1 Why the distinction matters

Every chapter that follows depends on this distinction, so it is worth being explicit about why it matters.

When you use AI to get answers, you are outsourcing the cognitive work that produces understanding. The AI is doing the thinking; you are doing the typing. Whatever the AI's answer is, you have not engaged with the structure of the problem in a way that builds your own ability to solve similar problems later. The next problem in this style will be just as hard for you as the first one, because you did not develop the relevant skill.

When you use AI to learn, you are using the AI as a Socratic interlocutor or a patient tutor. The AI generates explanations, alternative framings, counterexamples, edge cases, self-test questions—scaffolding around the problem. You then engage with the scaffolding and the problem yourself. You produce the answer. The cognitive work that produces understanding remains in your head, where it belongs.

The two patterns feel similar in the moment because both involve typing into an AI chat window. The difference shows up six weeks later, when you take an exam. The first student has covered material; the second student knows it. On the exam, the AI is not available.

1.2 A concrete contrast

Consider a student in a multivariable calculus course working on the following problem: *Compute the line integral of* $\vec{F}(x, y, z) = (yz, xz, xy)$ *along the curve* $\vec{r}(t) = (\cos t, \sin t, t)$ *from* $t = 0$ *to* $t = 2\pi$.

The answer-seeking approach: paste the problem into an AI tool. The tool returns the parameterization, the dot product, and the integral, with a numerical answer. The student copies the work into their homework. Time elapsed: 90 seconds.

The learning approach: ask the AI a different question. *"What is the conservative-vector-field test, and how do I apply it to $\vec{F} = (yz, xz, xy)$? What does it tell me about line integrals of this field?"* The AI responds with the test (check that the curl is zero), works the curl computation, finds it is zero, and explains that for conservative fields, the line integral depends only on the endpoints. The student now knows two things: the field is conservative, and they can compute the line integral by finding a potential function. The student then asks the AI to walk them through how to find a potential function for this specific field, but only the procedure—not the answer. They find the potential function, evaluate at endpoints, get the answer. Time elapsed: 12 minutes.

The first student has produced a homework submission. The second student has learned that this class of vector field has special structure, has practiced finding a potential function, and will recognize the pattern the next time they see it. The first student spent 90 seconds; the second student spent 12 minutes; on the next exam problem of this kind, the second student needs maybe two minutes and the first student is back at the AI chat.

This contrast extends to almost every kind of technical study. Across mathematics, physics, statistics, programming, and the sciences, the two ways of using AI lead to two qualitatively different learning trajectories. The rest of this guide is about how to consistently choose the second one.

1.3 The maxim

If you take only one habit from this chapter, take this one:

> *Before you ask an AI to solve a problem, ask it something else first.*

The "something else" is what turns AI from a homework completion engine into a study aid. The next chapter offers six concrete somethings. The chapter after that catalogs the failure modes you should be alert to. The chapter after that gives you the verification habit that protects you from accepting AI output that is plausible but wrong.

But the foundation is here, in this chapter. Use AI to learn, not to get answers. Everything else builds on this.

Chapter 2

Six Things AI Does Well

AI tools genuinely help with technical study in six categories. Each is a use mode that builds your understanding rather than substituting for it.

The previous chapter argued that there are two ways to use AI, and only one of them produces learning. This chapter is concrete about what that one looks like in practice. Six categories of AI use accelerate technical study, in ways that the literature on effective learning predicts will produce real and durable gains. Each gets a brief description, examples from several disciplines, and a short note on where the use mode goes wrong.

2.1 Clarifying notation and definitions

Technical fields use notation conventions that vary across textbooks, courses, and historical periods. A reader trying to follow a derivation in one source while reading the textbook from another routinely encounters symbols that don't match. AI tools are excellent at translating notation between conventions, defining unfamiliar symbols, and listing the alternate notations a concept goes by in different communities.

Cross-textbook notation translation

A statistics student is reading a paper that uses $\mathbb{E}[X|Y]$ for conditional expectation but their textbook uses $E_X(X|Y)$ and a different paper they have open uses $\langle X \rangle_Y$. They ask the AI: *"Are these three notations all expressing the same concept? List the contexts in which each is conventional."* The AI responds with the cross-conventions table. The student now reads all three sources without notation friction.

Disambiguating notation collisions

A physics student is reading classical mechanics and encounters L as the Lagrangian on one page and $\vec{L}$ as angular momentum two pages later. They ask: *"Is the use of L for both Lagrangian and angular momentum standard, and how do I tell which is meant in context?"* The AI explains the convention (the boldface or arrow distinguishes; the surrounding equations make context clear) and gives a few diagnostic phrases.

Notation as a window into a field's structure

A graduate student moving from physics into machine learning sees expectations written $\mathbb{E}_{x\sim p}[\, f(x)\,]$ and is confused by the subscripted distribution. They ask: *"Why does machine-learning notation make the sampling distribution explicit, when physics notation usually doesn't?"* The AI's response surfaces a substantive point about the field: in physics, the relevant distribution is usually fixed by the problem; in machine learning, the sampling distribution often *is* the parameter being optimized. The notation reflects the conceptual difference. The student now has a framing that will serve them across the rest of their reading.

Watch out

Notation translation is the use mode where AI is most reliable, but it is also where careless use most directly imports errors. If the AI tells you that $\nabla \cdot \vec{F}$ and $\operatorname{div} \vec{F}$ are the same thing, that is correct. If it tells you that two notations express the same concept but in fact one is a slightly different operator (gradient versus directional derivative, for instance), accepting the translation without checking against your textbook will produce wrong results in your work. Verify against a primary source.

2.2 Generating alternative explanations

A textbook presents one canonical explanation of a concept. Sometimes the canonical explanation lands; often it does not. Asking an AI for alternative framings of the same idea is one of the most consistently useful study habits available, because the framing that lands depends on the reader, and the textbook can only commit to one.

Three explanations of the central limit theorem

A statistics student is struggling with the central limit theorem (CLT). The textbook proof uses moment-generating functions; the student finds it opaque. They ask: *"Explain the central limit theorem in three ways: as a moment-generating-function result, as a geometric statement about averaging, and as the consequence of a mechanical convolution argument. Tell me which framing is most useful for building intuition."* The AI works through all three; the geometric framing (averages of any reasonable distribution converge to a Gaussian shape because Gaussian shapes are stable under convolution) lands for this student. They go back to the proof in the textbook with a much better grasp of what it's doing.

Alternative framings of recursion

A computer science student is learning recursion and finds the textbook examples (factorial, Fibonacci) artificial. They ask: *"Give me three different conceptual framings for what recursion is: as mathematical induction in disguise, as a compositional pattern, and as a stack-discipline mechanism. Use a worked example for each that isn't factorial or Fibonacci."* The AI provides three with examples (a tree-walk, a parser, a backtracking search). The student now has multiple mental models for recursion and can pick the one that matches the problem in front of them.

Watch out

Asking for "three different framings" sometimes returns three slightly-rephrased versions of the same framing. If that happens, push back: *"Those three are essentially the same. Give me framings that come from genuinely different traditions or use different mathematical machinery."* The AI's first response is often the easiest one to produce, not the most pedagogically useful.

2.3 Suggesting counterexamples and edge cases

Theorems in technical fields come with conditions. A student who understands a theorem deeply asks what happens when a condition is violated, where the boundary of the theorem's applicability lies, and what counterexamples exist if the conditions are weakened. AI tools are useful at generating candidate counterexamples and edge cases for further investigation.

Counterexamples to a misremembered theorem

A student remembers that "differentiable functions are continuous" but is unsure of the converse. They ask: *"Is every continuous function differentiable? If not, give me a counterexample."* The AI provides $f(x) = |x|$ at $x = 0$ as the standard counterexample, then offers the more striking Weierstrass function (continuous everywhere, differentiable nowhere). The student now has a clear picture of how the implication runs in only one direction.

Edge cases for a numerical method

A computational student is implementing Newton's method for root-finding and asks: *"Generate three failure modes for Newton's method that aren't in the standard textbook list. For each, give me a concrete function and starting point that exhibits the failure."* The AI returns three: divergence near a flat tangent, oscillation between two basins of attraction in a function with closely-spaced roots, and sensitivity to step size in a stiff regime. The student tests each, sees the failure, and now understands the method's robustness limits in a way the textbook didn't convey.

Stress-testing a statistical method

A data-science student is using bootstrap confidence intervals for a difference of means. They ask: *"In what data settings will the bootstrap confidence interval undercover the true parameter? Give me three settings I should test on simulated data."* The AI lists heavy-tailed distributions, small sample sizes with skewed populations, and dependent data. The student simulates each, observes the undercoverage, and now uses the bootstrap with awareness of when to trust it.

Watch out

AI-generated counterexamples are sometimes wrong—the proposed counterexample doesn't actually violate the theorem, or the failure mode for a numerical method doesn't actually occur on the proposed example. Treat AI-generated counterexamples as candidates to be checked, not as established results. The chapter on verification (Chapter 4) develops this further.

2.4 Computational prototyping

Writing code from scratch is time-consuming. AI tools can sketch a workflow's structure quickly, leaving the reader to verify the mathematics, test the implementation, and repair the parts the AI got wrong. Used as a prototyping accelerator, AI compresses the early-stage work of getting a workflow running while the substantive thinking remains the student's.

Sketching a regression workflow

A statistics student needs to run a multivariable regression with diagnostic plots. They ask the AI to sketch the workflow in their preferred language (Python, R, MATLAB), then they go through the sketch line by line and verify each step against what their textbook says the workflow should do. The student adjusts a few defaults (the AI used unweighted least-squares; the data calls for weighted), corrects one library import that was for a deprecated package, and adds a residual plot the AI omitted. The workflow runs cleanly. Total time: under an hour, of which 10 minutes was spent typing.

Prototyping a simulation

A physics student needs to simulate a damped harmonic oscillator with a forcing function. They ask the AI for a basic numerical-integration sketch. The AI returns a Runge-Kutta-4 implementation. The student verifies the equations of motion against their notes, checks the time-stepping logic, runs

the simulation, and inspects the energy curve. The energy isn't conserved as expected; the student investigates and finds the AI used a non-symplectic integrator. They switch to the appropriate symplectic method, knowing now the difference matters for this kind of system. In the process, they have learned why integrator choice matters in mechanics simulations—a lesson the original homework problem didn't directly teach.

Watch out

The most common AI failure mode in computational prototyping is not bad code; it is plausible code that implements subtly wrong mathematics. The code runs, produces output, and the output looks reasonable, but the implementation differs from what the assignment or the analysis required. Chapter 3 develops this failure mode at length, and Chapter 4 provides verification techniques.

2.5 Identifying what to study

A student opening a difficult chapter often does not know what background they need before reading it. Asking the AI to inventory the prerequisites is a fast way to identify whether the chapter is approachable now or whether some review is needed first.

Inventorying prerequisites

A student is about to read a chapter on quantum harmonic oscillators in a physical-chemistry textbook. They ask: *"List the mathematical and physical prerequisites for this chapter, with one-sentence descriptions of each. Identify which I should review before reading and which the chapter probably re-introduces."* The AI lists differential equations (review needed), Hermite polynomials (chapter probably reintroduces), expectation values (the student has these from a previous course), and ladder operators (the chapter introduces). The student spends 20 minutes reviewing differential equations before reading the chapter, and the chapter is now tractable. Without the inventory, they might have stalled three pages in.

Identifying weak spots in advance of an exam

A student preparing for a final asks: *"Given the topics covered in chapters 1 through 8 of this textbook, what are the three concepts that students most often get wrong on exams of this material? For each, give me a self-test question."* The AI lists three (with self-test questions). The student attempts the questions, finds they get one mostly right and two wrong, and focuses their remaining study time on the two areas where they were weak. The exam is on Tuesday; without the AI, they might have spent the same time re-reading chapters they already understood.

Watch out

The AI's prerequisite list is a candidate, not an authority. Sometimes the AI lists prerequisites that aren't actually needed (because the textbook reintroduces them), and sometimes it omits a prerequisite that is needed (because the prerequisite is implicit in the textbook's intended audience). Treat the list as a starting point for your own judgment about what to review.

2.6 Self-testing through retrieval practice

This is the use mode with the strongest empirical support in the cognitive-science literature, and the one most students underuse. Asking an AI to pose questions about material you have just studied—and then closing the AI tab and answering the questions in writing or aloud—is one of the most effective study techniques available. Chapter 5 discusses why; this section covers the mechanics.

Self-test on a chapter just read

A student has just finished reading a chapter on the Krebs cycle in a biochemistry textbook. They ask: *"Pose five questions about this chapter that an oral-exam committee might ask. Range from straightforward fact retrieval to questions that require connecting the material to broader metabolism."* The AI lists five questions. The student closes the chat window, writes answers on paper, then opens the chat to compare. They find their answers were strong on the citric-acid mechanism but weak on how the cycle's outputs feed into the electron transport chain. They reread the relevant pages with a clear sense of what they didn't actually know.

Self-test as exam preparation

A student preparing for a calculus exam asks: *"Generate a 30-minute mock exam covering the topics from this week's lectures. Mix easy and hard questions; include at least one question that requires recognizing which technique applies, not just executing a known technique."* The AI generates the exam. The student takes it under timed conditions, then reviews their answers without the AI's help, then asks the AI to grade and explain. The exercise finds two weak spots they can repair before the actual exam.

Self-test in a programming context

A student learning a new programming language asks: *"Generate three coding problems on the topics I just studied, of increasing difficulty. Don't show me the solutions yet."* They solve all three, then ask the AI to review their solutions. They find one solution worked but for the wrong reason; the AI's review surfaces this and they fix the underlying misunderstanding before it appears in their next assignment.

Try this

After you finish this chapter, before reading further, do this: list the six categories from memory, and for each one, write down (a) what the category is, (b) one example of using AI in that category, and (c) one watch-out. Then open the chapter and check your list. The act of trying to retrieve the information is more valuable than the act of re-reading the chapter, and you'll find the categories you didn't quite remember are the ones worth re-reading.

2.7 What the six have in common

A pattern runs through the six categories. In every case, the student is using AI to surround the problem with structure—definitions, alternative framings, counterexamples, prototypes, prerequisites, self-tests—rather than asking AI to solve the problem itself. The AI is doing what a patient and tireless tutor would do: building scaffolding around the problem and helping the student climb it. The cognitive work—the part that produces understanding—stays with the student.

This is the same maxim from the previous chapter, made concrete in six ways. Use AI to learn, not to get answers; here are six things to ask for instead.

Chapter 3

Six Things AI Does Badly

The same tools that scaffold learning also produce confidently-stated nonsense. Six failure modes recur. Knowing them by name is the first step to catching them.

This chapter is the counterpart to the previous one. It catalogs six failure modes that AI tools exhibit reliably across versions and across vendors. The list is not exhaustive—new failure modes will emerge as AI capabilities evolve—but the six listed are the ones that have shown up repeatedly in technical work over multiple AI generations and that are most likely to bite a student who is not watching for them.

The chapter's structure is uniform: each failure mode gets a name, a description, a worked example showing how the failure looks in practice, and a verification heuristic for catching it.

3.1 Citation hallucination

What it is. The AI invents references. Author names, journal names, paper titles, and years can all be approximately right while the cited paper does not exist or does not say what the AI says it says. The pattern is most common when the AI is asked to support a specific claim with a citation; it generates a plausibly-located reference rather than retrieving an actual one.

A fabricated reference for a real result

A student writing a literature review on epidemic modeling asks the AI: *"Give me a citation for the original derivation of the SIR model."* The AI returns: *"Kermack, W.O., & McKendrick, A.G. (1927). A Contribution to the Mathematical Theory of Epidemics. Proceedings of the Royal Society of London A, 138(834), 25–47."* The authors and year are correct. The journal volume and pages are subtly wrong (the actual paper is in volume 115, not 138). The student includes the citation in their bibliography. A reviewer notices the volume number is wrong and the student loses credibility.

A confidently-stated nonexistent paper

A graduate student writing a research proposal asks the AI for a recent paper supporting a methodological claim. The AI returns a citation with author names, journal name, year, and page numbers, all plausible. The student tries to look up the paper to read it before citing. The paper does not exist; the authors are real and have published on related topics, but not this specific paper. The student avoids a serious credibility problem because they tried to retrieve the source before citing.

Verification heuristic. *Never cite a paper you have not retrieved and read at least the abstract of, even when the AI returns the citation in standard format.* Every citation in any AI-assisted work must be independently verified against a primary database (Google Scholar, Web of Science, the publisher's website, or the journal's archive). If a citation cannot be retrieved, it does not exist; revise your work to use a citation you have actually read.

3.2 Unverified analysis acceptance

What it is. The AI carries out a derivation that contains a wrong step but reaches a plausible-looking conclusion. The intermediate error is subtle enough that a reader skimming the derivation does not notice; the conclusion is plausible enough that the reader does not check.

A wrong step that looks right

A student in a probability course asks the AI to derive the variance of the sum of two independent random variables. The AI writes: $\mathrm{Var}(X+Y) = E[(X+Y)^2] - (E[X+Y])^2$, expands the square, distributes the expectation, and reaches $\mathrm{Var}(X+Y) = \mathrm{Var}(X) + \mathrm{Var}(Y) + 2\,\mathrm{Cov}(X,Y)$. The conclusion is correct in general, but the AI's derivation didn't use the independence assumption that the question stipulated. For independent X and Y, the covariance term is zero, and the derivation should have stated this. The student copies the result without noticing that the derivation didn't use a key hypothesis. On the next problem, where the variables are not independent, they apply the simplified formula and get the wrong answer.

Wrong because of a sign error in the middle

A physics student asks the AI to derive the equation for the period of a simple pendulum from the small-angle approximation of the equation of motion. The AI sets up $\ddot{\theta} + (g/L)\sin\theta = 0$, applies the small-angle approximation $\sin\theta \approx \theta$, and reaches the simple-harmonic-oscillator form, then derives $T = 2\pi\sqrt{L/g}$. Along the way, the AI's intermediate step had the gravitational acceleration with the wrong sign in one line, but the final answer is correct because the sign error is squared in the period formula. The student reading the derivation absorbs the wrong sign convention and applies it on the next problem (a vertical-spring oscillator) where the sign error matters. They get the wrong answer there.

Verification heuristic. *Read derivations as if they were written by a careful but fallible human, with errors scattered through them.* Check the units of every equation; check that every assumption stated in the problem is used somewhere in the derivation; spot-check at least one numerical instance to catch algebraic errors. The verification chapter (Chapter 4) develops these techniques.

3.3 Meaning drift in technical prose

What it is. The AI returns prose that subtly changes the meaning of the question's technical claim. Quantifiers become weaker ("all" becomes "most"), hedging is added or removed, scope conditions shift. The student who copies the AI's prose ends up making a claim subtly different from the one they meant.

A quantifier softening

A student asks: *"Explain why all continuous functions on a closed bounded interval attain their maximum."* The AI's response begins: *"Continuous functions on closed bounded intervals typically attain their maximum because. . . "* The word "typically" is wrong—the extreme value theorem applies to *all* continuous functions on closed bounded intervals, not most. A student who copies the AI's framing into their own work has weakened a theorem.

Hedging that wasn't there

A student writing a statistics report asks the AI to summarize a finding that says "treatment A reduces mortality by 15%." The AI rephrases as "treatment A may reduce mortality by approximately 15%." The hedge is plausible but not in the source. If the student copies the AI's prose, their report claims something weaker than the source supported. If the student's audience compares the report against the source, the discrepancy looks like overcaution; if the audience trusts the report, they may underestimate the treatment's effect.

Verification heuristic. *When using AI to rephrase a technical claim, compare the rephrasing word-by-word against the original.* Quantifiers (*all, every, most, some, none*) are the highest-leverage words; check them first. Hedging language (*may, might, typically, often, possibly*) is the next-highest-leverage;

if the AI added it, ask whether your source actually warrants the hedge. Adopt only the rephrasing that preserves the original's logical force.

3.4 Code that runs but does the wrong thing

What it is. The AI generates code that compiles, runs without errors, and produces output, but implements mathematics that differs from what the student needed. The code's apparent correctness (it didn't crash) masks its substantive incorrectness (it computed something else).

Wrong update rule in numerical optimization

A student implements gradient descent for a convex function and asks the AI to write the optimization loop. The AI returns a loop that updates the parameter as $\theta_{t+1} = \theta_t + \eta \nabla f(\theta_t)$ instead of $\theta_{t+1} = \theta_t - \eta \nabla f(\theta_t)$. The code runs without errors. The optimization diverges. A student who runs the code only briefly and sees that the loss is increasing might conclude their function is non-convex (mistakenly) rather than that the sign in the update is wrong.

Off-by-one error in a probabilistic simulation

A student simulating a random walk asks the AI for a Monte Carlo estimate of the probability of returning to the origin within n steps. The AI's code samples n random steps, but the loop counts a return at step 0 as a return (when in fact the walk hasn't moved). The simulation returns probability 1 for any $n \geq 0$. The error is invisible from output alone; only by inspecting the loop logic does the student catch it.

Different-but-runnable statistics

A student running a regression asks the AI for code to compute the standard errors of the regression coefficients. The AI returns code that uses the homoskedastic standard-error formula, but the student's data are heteroskedastic and they meant to use robust (sandwich) standard errors. The code runs and produces standard-error values; the values just happen to be the wrong ones. The student's downstream confidence intervals are wrong by a factor that depends on the heteroskedasticity.

Verification heuristic. *Code is not verified by running it; code is verified by reading it and by checking its output against an independent calculation.* For each AI-generated piece of code, (a) read line by line and ask whether each line does what your problem requires, (b) compute one instance of the calculation by hand or in a different tool, and check that the code reproduces the hand-computed value, and (c) test on a known special case where you can predict the output. The verification chapter develops these checks.

3.5 Source conflation

What it is. The AI assembles a response by drawing on multiple sources from its training data, mixing notation, conventions, or framework choices that the source documents kept separate. The resulting response is internally inconsistent in ways that are hard to spot if the reader doesn't already know the source documents.

Mixed notation conventions

A student asks the AI to derive Maxwell's equations in a unified notation. The AI's response is correct mathematically but mixes Gaussian units in some equations and SI units in others, with the unit-system choice changing line by line. The equations are individually right; together they are nonsense, because a derivation cannot use two unit systems concurrently. The student copying the

response into their notes will encounter contradictions when they try to use the equations together.

Mixed framework conventions in machine learning

A student studying reinforcement learning asks the AI to explain Q-learning. The response uses one paper's discount-factor convention (γ applied to rewards from time $t+1$ forward) and another paper's value-function definition (which absorbs γ into the value function itself). The two conventions produce algebraically different expressions; mixing them makes the explanation incoherent. A reader who knows neither convention will not catch the inconsistency.

Verification heuristic. *Pick a single source—your textbook, a single paper, a single convention—and translate any AI response back into that source's notation and conventions before using the response.* If the translation introduces inconsistencies, the AI was conflating sources, and the response needs to be rewritten cleanly in your chosen convention.

3.6 Multi-language divergence

What it is. The AI produces working implementations in two programming languages (or two software environments) that should compute the same thing but actually behave differently because of language-specific defaults the AI did not flag.

Default integer division

A student asks the AI to implement the same algorithm in Python and in C. In Python, integer division uses `//`; `/` returns a float. In C, integer division of two integers truncates. The AI's two implementations use `/` in both cases. In Python, the algorithm produces correct results because `/` returns float division; in C, the same code produces wrong results because the division truncates. The discrepancy is invisible until the student runs both and notices the answers differ.

Different random-seed behavior

A student implements a Monte Carlo simulation in R and in MATLAB, intending the two implementations to produce reproducible identical results given the same seed. The AI's two implementations both call `set.seed(42)` or `rng(42)`, but the random number generators in R and MATLAB are not the same algorithm; the same seed produces different sequences. The student concludes the implementations differ algorithmically when in fact they only differ in their underlying RNG.

Verification heuristic. *When using AI to produce parallel implementations in two languages, run both on a small test case and compare outputs to floating-point precision. If outputs differ, ask the AI to identify the source of the discrepancy.* The discrepancy is usually a language default the AI implicitly relied on without flagging; once flagged, it is easy to fix.

3.7 The shape of these failures

A pattern runs through the six failure modes. In every case, the AI's output is plausible—it looks like what a competent person would produce—but contains an error that makes it incorrect. The errors are not random; they follow predictable patterns that a student can learn to anticipate.

The next chapter develops the verification habit that catches these failures most efficiently. Reading that chapter is the single most important thing you can do to use AI safely in technical study.

Chapter 4

The Verification Principle

Never accept AI output without independent verification. The principle is short. The chapter is about how to apply it without losing the speed advantage that made you reach for AI in the first place.

Both previous chapters contain the same implicit message: AI tools produce output that looks correct and sometimes isn't, and the student who accepts the output uncritically is the student who learns the wrong things. The remedy is a habit, not an attitude. The habit is verification, and this chapter is about how to make it efficient.

The chapter has three sections, one for each of the most useful verification techniques in technical work. The techniques are general—they apply across mathematics, physics, statistics, programming, and the sciences—but each works best in particular contexts. After the three techniques, a concluding section introduces a checklist that synthesizes them into a usable workflow.

4.1 Dimensional verification

The most efficient verification technique in any quantitative field is checking that the units are consistent. Every physical or mathematical equation has units on both sides; the units must match. AI-generated derivations frequently fail this check, and the failure is fast and unambiguous to detect.

Catching a wrong formula in physics

A student asks the AI for the formula relating period T, length L, and gravitational acceleration g for a simple pendulum. The AI returns $T = 2\pi\sqrt{L \cdot g}$. The student checks units: L is in meters, g is in $\mathrm{m/s^2}$, so $L \cdot g$ has units of $\mathrm{m^2/s^2}$, and the square root has units of m/s. But T should be in seconds. The formula is wrong; the correct version is $T = 2\pi\sqrt{L/g}$, which gives units of seconds. The dimensional check caught the error in 30 seconds without the student needing to remember the formula.

Catching a wrong formula in chemistry

A student asks the AI for the relationship between Gibbs free energy ΔG, enthalpy ΔH, entropy ΔS, and temperature T. The AI returns $\Delta G = \Delta H + T \cdot \Delta S$. The student checks: if ΔH is in joules and $T \cdot \Delta S$ has units of $\mathrm{K} \cdot \mathrm{J/K} = \mathrm{J}$, the equation is dimensionally consistent. But the physical meaning is wrong (the formula should have a minus sign: $\Delta G = \Delta H - T \cdot \Delta S$). Dimensional verification didn't catch it—the units check out either way. This example illustrates a limitation: dimensional verification catches many errors but not all.

Catching a wrong formula in statistics

A student asks the AI for the relationship between the standard deviation of a sample mean and the standard deviation of the underlying population. The AI returns $\sigma_{\bar{X}} = \sigma_X \cdot n$, where n is the sample size. The student checks: if σ_X has units of (the data's units) and n is dimensionless, then $\sigma_{\bar{X}}$ has the same units as σ_X. The dimensional check passes. But the formula is wrong; the correct version is $\sigma_{\bar{X}} = \sigma_X/\sqrt{n}$. Again, dimensional verification didn't catch this one. (A different verification—taking n to infinity—would have: the AI's formula gives unbounded standard deviation as the sample size grows, which is the opposite of what should happen.)

The technique is fast (seconds to minutes per equation) and catches a real fraction of common errors, but it is not a complete check. The next two techniques fill in the gaps.

Try this

The next time you receive an AI-generated equation in any quantitative field, write the units of every variable above its first appearance and verify the equation. If the units don't match, the equation is wrong; if they match, the equation might still be wrong but is not wrong in the most common way.

4.2 Special-case reduction

A general result must reduce to known special cases when its parameters take special values. AI-generated derivations frequently fail this check, and the failure is unambiguous. The technique is most useful in mathematics and theoretical physics, where general results are often built up from special cases the student already knows.

Reducing a general statistical result

A student is given an AI-derived formula for the variance of a weighted sum of correlated random variables. They check the special case where the variables are uncorrelated and the weights are all equal: the formula should reduce to σ^2/n (variance of an average). It doesn't. The AI's general formula has an extra factor of n that shouldn't be there. The student goes back and finds a missing $1/n^2$ in one of the AI's intermediate steps.

Reducing a general physics result

A student is given an AI-derived expression for the kinetic energy of a relativistic particle. They check the special case where the particle's velocity is much less than the speed of light: the formula should reduce to $\frac{1}{2}mv^2$. It does, but only after the student carefully expands the AI's expression in a Taylor series. The reduction confirms the AI's formula is correct in this case.

Reducing a numerical method

A student is given an AI-generated stencil for a finite-difference approximation of a second derivative. They check the special case where the function is a quadratic polynomial: the stencil should give the exact second derivative for any quadratic. The AI's stencil gives something else. The student finds the AI used coefficients $(1, -1, 1)$ instead of $(1, -2, 1)$. The stencil is wrong; the special-case check caught it instantly.

The technique works whenever there is a simpler version of the result the student already knows. For results without obvious special cases, the next technique is the workhorse.

4.3 Numerical spot-check

A single numerical instance, computed independently, catches most algebraic errors. The technique is simple: pick a set of parameter values, compute the result the AI gave you in those parameter values, compute the correct result independently (by hand, by a different tool, or against a known table), and compare.

Catching an algebraic error in a derivative

A student asks the AI for the derivative of $f(x) = \ln(x^2 + 1)$. The AI returns $f'(x) = \frac{1}{x^2+1}$. The student tests at $x = 1$: the correct derivative there should be $\frac{2}{2} = 1$; the AI's expression gives $\frac{1}{2}$. The

numerical spot-check catches the missing chain-rule factor of $2x$ in 30 seconds. The correct derivative is $f'(x) = \frac{2x}{x^2+1}$.

Catching a wrong combinatorial formula

A student asks the AI: *"How many ways are there to arrange the letters in the word "MISSISSIPPI"?"* The AI returns a number. The student verifies by computing the formula explicitly: $\frac{11!}{4!\cdot 4!\cdot 2!} = \frac{39916800}{1152} = 34{,}650$. If the AI's number matches, the answer is verified; if not, the student knows to look for an error.

Catching a wrong matrix-inversion result

A student asks the AI to invert a 2×2 matrix symbolically. They verify by computing $A \cdot A^{-1}$ for a specific numerical matrix and checking that the product is the identity. If the product isn't the identity, the AI's symbolic inversion has an error somewhere; the student then localizes the error by inspection.

The technique is general: it works in any field where you can compute a specific instance of a general result. The cost is a few minutes per check; the benefit is catching the silent algebraic errors that dimensional and special-case checks miss.

4.4 A generic verification checklist

The three techniques above can be combined into a checklist that applies to almost any AI-generated technical content. The checklist is six items long; running through it takes a few minutes for most pieces of work.

The Generic Verification Checklist

For every piece of AI-generated technical content—a derivation, a code snippet, a literature summary, a worked example—ask the following six questions before accepting it:

1. *Are the units consistent on both sides of every equation?*
2. *Does the result reduce to a known special case under appropriate parameter limits?*
3. *Does a single numerical instance, computed independently, agree with the AI's result?*
4. *Are all the assumptions stated in the question used somewhere in the derivation or analysis?*
5. *If a citation is given, can I retrieve the cited source and verify what it actually says?*
6. *If code is given, can I read it line by line and confirm each line implements what I intended?*

A piece of work that passes all six checks is not necessarily correct—no finite set of checks can guarantee correctness—but it is at least internally consistent in the most common ways. A piece of work that fails any check is a draft to repair, not a result to use.

The checklist is in Appendix A in pocket form, suitable for printing and keeping next to your desk. The act of running through it explicitly, rather than relying on the feeling that something is right, is what separates students who use AI safely from students who do not.

4.5 Why verification feels expensive but isn't

The most common objection to verification is that it slows you down. If you ran every AI response through a six-item checklist, you would spend as much time verifying as you would have spent doing the work yourself. The objection is partly right and partly wrong, and the part that's right matters.

The objection is partly wrong because verification is much faster than doing the work from scratch. A dimensional check takes 30 seconds; a numerical spot-check takes a few minutes; a code review takes longer but is still faster than writing the code. The total verification time is usually 10–20% of the time it would have taken to do the work without AI help.

The objection is partly right because verification has a fixed minimum cost, and for very small tasks, the cost is comparable to just doing the task. Asking the AI to evaluate a single integral and then verifying its answer is not faster than evaluating the integral yourself. The break-even point is around the level of a multi-step problem; for tasks below that, AI use is genuine acceleration only when verification is built in.

The lesson is to use AI selectively. For tasks where the AI compresses substantial work and verification is fast (most derivations, most code prototypes, most literature questions), the verification habit pays for itself many times over. For tasks where the AI's work is comparable to your own time (single equations, brief computations, factual lookups you could find faster in a textbook), AI use without verification is a recipe for absorbing wrong answers, and AI use with verification offers no time savings. Don't use AI for those tasks; do the work yourself.

The student who applies this judgment well uses AI on the right tasks and verifies on every task they use it for. The student who applies it badly uses AI on every task and verifies on none of them. The first student gets faster, more accurate, and more learned over the course of a semester. The second student gets stuck.

Chapter 5

Cognitive Offloading and the Evidence

The previous four chapters argue that you should use AI to learn, not to get answers, and that you should verify everything. This chapter explains why—grounded in what cognitive science actually says about how learning works.

The argument so far has been presented as a set of practical recommendations. It is also an empirical claim about how human learning works. The recommendations follow from a body of research on memory, retrieval practice, productive failure, and cognitive offloading that has been steady for decades. This chapter summarizes that research and connects it to the recommendations. The summary is short by design; the references at the end of the guide point to fuller treatments.

If you are skeptical that AI use can quietly damage learning, this is the chapter that should change your mind—or at least make the cost concrete. If you are already convinced, this chapter gives you the language to explain the position to other students who are not.

5.1 Retrieval practice: why effortful recall outperforms re-reading

The single most consistent finding in the cognitive-science literature on learning is that *retrieval practice*—trying to recall information from memory, especially when the recall is difficult—produces stronger long-term retention than rereading or reviewing the same material passively. The finding is robust across age groups, subject areas, and laboratory designs (Karpicke and Roediger 2008; Carpenter et al. 2022; see Appendix C for full references).

The mechanism is not mysterious. When you successfully retrieve a piece of information, the act of retrieval strengthens the memory and the cues that led to it. When you fail to retrieve something and then look it up, the failure-then-recovery sequence makes the information more likely to be recalled the next time. When you simply reread, neither effect occurs; the material feels familiar but the memory is not strengthened.

The implication for AI use is direct. Asking the AI to explain a concept that you have just struggled to recall is retrieval practice—you struggled, you failed, and you used the AI to recover the information you could not produce on your own. The act of struggling produced the strengthening effect; the AI just helped you finish.

Asking the AI to explain a concept *before* you have tried to recall it is something different. There has been no retrieval attempt. The strengthening effect that retrieval practice produces does not occur. You have substituted recognition for recall: you read the AI's explanation and recognize it as correct, but you have not built the cue structure that would let you recall the same information yourself two weeks later.

Example

A student studying for an exam in linear algebra asks the AI: *"Explain the spectral theorem and give me an example of how to apply it."* The AI provides a clean explanation. The student reads it, finds it makes sense, and moves on.

Two weeks later, the student takes the exam. A question asks them to explain when the spectral theorem applies and to apply it to a specific matrix. The student remembers the AI explained this clearly, but cannot reconstruct the explanation on demand. They write something approximate and

lose most of the credit.
The same student, using a retrieval-practice approach, would have started by trying to recall the spectral theorem from their notes. They would have failed, partly. They would have asked the AI to fill in the gap, then closed the AI tab and tried again to state the theorem from memory. They would have succeeded the second time. On the exam, they would recall the theorem because they had practiced recalling it.

The first student's hour with the AI was a comprehension exercise; the second student's hour was a learning exercise. The activities looked similar. The outcomes were not.

5.2 Productive failure: why struggling first beats being told

A related finding: students who struggle to solve a problem before being given the solution method learn the method more deeply than students who are taught the method first (Kapur 2014; Loibl et al. 2017). The result is counterintuitive—surely it would be more efficient to learn the method first and then practice it?—but the empirical evidence is clear. The struggle, even when it ends in failure, primes the student to engage with the eventual solution in a way that direct instruction does not.

The mechanism is again not mysterious. When you struggle with a problem, you generate a structure of attempted approaches, near-misses, and recognized obstacles. This structure becomes the scaffolding into which the eventual solution method is fitted. The solution then makes sense because you understand what problem it is solving. A student who learns the method first has no such scaffolding; the method is a procedure to be memorized rather than a tool that solves a problem the student already cares about.

Example

Two physics students are learning Lagrangian mechanics. The first student opens the chapter, notes that the Lagrangian is $L = T - V$ (kinetic minus potential energy), reads the derivation of the Euler-Lagrange equations, and works through the worked examples. The chapter feels straightforward. The student finishes the chapter in 90 minutes and moves on.
The second student opens the chapter, sees the Lagrangian definition, and is puzzled: why this combination? Why not $T + V$ or some other combination? Before reading further, the student spends 20 minutes trying to derive Newton's second law from $L = T - V$ and from a few alternative candidates, and discovers that only $T - V$ produces the right equations of motion. They then read the chapter's derivation and find it makes deep sense—the Euler-Lagrange equations are the conditions under which the action is stationary, and only $T - V$ gives an action whose stationary points are the solutions of Newton's law.
Both students have spent 90 minutes. The first student has read the chapter; the second has understood it. On the next exam, the second student is more likely to apply the method correctly to an unfamiliar problem because they understand what problem the method solves.

The connection to AI is again direct. Asking the AI to explain a method before you have struggled with the underlying problem skips the productive-failure phase. The explanation lands on no scaffolding; it is absorbed at the level of recognition rather than understanding.

The recommendation that follows: *when you encounter a new method or concept, try the underlying problem yourself for at least 15 minutes before consulting AI for help.* Fifteen minutes of struggle is enough to build scaffolding; fifteen minutes is also short enough that the cost of failure is bounded. The struggle does not need to succeed; it needs to be genuine.

5.3 The illusion of fluency

Both retrieval practice and productive failure share a common diagnostic: they require effort, and effort feels like learning is going badly. The effortful struggle to recall information *feels* less productive than the smooth reading of an AI-generated explanation. The 20 minutes of failed attempts at Lagrangian mechanics *feels* less productive than the 20 minutes spent reading a clear chapter.

Cognitive scientists call this the *illusion of fluency*: study activities that feel smooth and easy produce a sense of comprehension without producing the durable memory traces that distinguish comprehension from learning (Bjork 1994; Kornell and Bjork 2008). Students systematically underrate study activities that feel difficult and overrate those that feel easy, even when objective tests of learning show the opposite ranking.

AI tools are dangerously suited to producing the illusion of fluency. An AI explanation is, almost by design, smooth and clear. Reading it produces a strong sense of understanding. The understanding is real *at the moment of reading* and largely fictitious *a week later when the explanation is no longer in front of you.* A student who relies on AI explanations as their primary study mode trains themselves to feel they have learned material that they have not in fact retained.

The remedy is to distrust the feeling of fluency and to trust the feeling of difficulty. If a study session feels easy, it probably is not building durable memory. If it feels effortful, it is.

Try this

A practical test for whether your study session is building learning or just producing the feeling of it: at the end of the session, close every reference (textbook, notes, AI tool) and write down what you learned, in your own words, without consulting anything. If you can do this comfortably, the session built durable memory. If you cannot, the session produced the illusion of fluency. The latter is not necessarily wasted—it may have been useful preparation—but it has not yet built the kind of memory that an exam will test.

5.4 Cognitive offloading: when outsourcing skills permanently weakens them

The phenomena above are about study sessions. There is a longer-term phenomenon at the level of skill development: skills that you consistently outsource to a tool atrophy or fail to develop, and the atrophy persists even when the tool is unavailable. This is *cognitive offloading* (Risko and Gilbert 2016).

The classic examples predate AI. Drivers who rely on GPS navigation lose the ability to read maps and develop poorer mental models of the cities they drive in. Students who rely on calculators for arithmetic lose facility with mental computation. Phone-number memorization—once a basic adult skill—has effectively disappeared as a result of contact-list autofill. The pattern is general: a tool that performs a cognitive task efficiently substitutes for, rather than augments, the underlying skill.

The same pattern applies to AI use in technical study. A student who consistently asks AI to perform derivations does not develop derivation skill, even if they understand each AI-produced derivation when they read it. A student who consistently asks AI to write code does not develop coding skill, even if they understand each AI-written program. A student who consistently asks AI to summarize papers does not develop the skill of reading carefully, even if they understand each summary.

The cost compounds across a degree. The first cohort of students to use AI extensively in their coursework will graduate without skills that previous cohorts considered foundational—and without realizing they lack the skills, because the AI was always there to fill the gap. The first time many of these students will discover the gap is on a job interview, a technical screen, a comprehensive exam, or a research crisis where AI assistance is unavailable, insufficient, or visibly produces wrong answers.

The recommendation that follows from cognitive-offloading research: *periodically perform cognitive tasks without AI, even when AI is available, so the underlying skills develop and are maintained.* The schedule does not need to be heroic; one or two no-AI study sessions per week, in each of your core courses, is enough to keep the underlying skills active.

5.5 What this means in practice: specific habits

The four lines of evidence above—retrieval practice, productive failure, the illusion of fluency, and cognitive offloading—converge on a set of specific study habits. The habits are not new; they are what good students have always done. The presence of AI tools makes the habits both more important (because the alternative is more attractive) and more difficult to maintain (because the alternative is more attractive).

- *Try the problem before consulting the AI.* Fifteen minutes of struggle, even unsuccessful struggle, builds the scaffolding that lets the eventual explanation land.
- *Use AI for self-testing rather than for answer-checking.* Have the AI generate questions that you answer without consulting AI. Use AI's evaluation only after you have committed your own answer.
- *When AI explains something, close the AI tab and try to reproduce the explanation in your own words.* If you cannot, you absorbed at the level of recognition, not understanding. Repeat until you can.
- *Schedule periodic no-AI study sessions* in each of your core courses. The exact frequency matters less than the consistency.
- *Distrust the feeling of fluency.* Sessions that feel easy probably are not building durable memory. Sessions that feel hard probably are.
- *Distinguish comprehension from learning* when you assess your own progress. Comprehension is when you understand the material in front of you. Learning is when you can reproduce the understanding without the material.

These habits are not optimizations. They are the difference between students who finish a degree and have learned the material and students who finish a degree and have, at most, made fluent contact with it. The cost of the second outcome is invisible until you need the skills you didn't build—which, given a long career, you will.

Chapter 6

Disclosure and Intellectual Honesty

If you used AI to help produce some piece of work, your reader should know. The principle is uncontroversial; the practice is harder than it looks.

Disclosure of AI use is a practical and ethical requirement that most institutions are still figuring out. Different courses have different policies; different journals have different requirements; different employers have different expectations. The practical challenge for a student is to develop a disclosure habit that works across all these contexts and that you can maintain without thinking too hard about it.

This chapter offers a five-field disclosure template that has worked well across courses and assignment types, three worked examples showing how the template applies to different kinds of work, and a discussion of when disclosure matters and when it does not.

6.1 Why disclosure matters

Three reasons. The first is straightforward: in many contexts, undisclosed AI use is a violation of academic-integrity policy. Even where it is not formally a violation, it usually is morally a violation—the work you submit is being represented as more your own than it is.

The second is professional. In research, undisclosed AI assistance is not yet uniformly forbidden, but it is increasingly required to be disclosed. A paper that uses AI assistance and does not disclose it is more vulnerable to retraction than a paper that uses the same assistance and discloses it. Disclosure protects you and your collaborators; non-disclosure exposes you both.

The third is pedagogical. The act of writing a disclosure is itself useful—it forces you to articulate what you used AI for, what you accepted, what you rejected, and what you verified. The articulation is a small additional learning act each time you do it, in the same way that writing a study summary is a small additional learning act on top of the studying itself. Students who disclose well tend, in my experience, to use AI better, because the disclosure habit makes them watch what they do.

6.2 The five-field disclosure template

A disclosure that covers the substantive cases needs five fields. Each is short—a sentence or two. Together they take perhaps five minutes to write at the end of a typical assignment.

The Disclosure Template

1. *What you asked.* The question or prompt you posed to the AI, in summary. Verbatim is best, but a faithful paraphrase is acceptable for short prompts.
2. *What it returned.* A brief summary of the AI's response. You do not need to quote it in full; you need to characterize it accurately enough that a reader knows what was on the table.
3. *What you accepted, in your own words.* The parts of the response that ended up in your work, restated as you understand them. This is the most important field: it forces you to demonstrate that you understood what you used.

4. *What you rejected, with brief reason.* The parts you decided not to use, and why. This shows you exercised judgment rather than passively accepting.
5. *What you verified independently and how.* The checks you performed on the parts you accepted. This is the field that distinguishes scholarly from credulous AI use.

The template is in pocket form in Appendix B for easy reference. The five fields cover the cases that actually arise: a derivation you used, code you wrote, a citation you found, a phrase you adapted, an interpretation you accepted.

6.3 Three worked examples

The template applies differently to different kinds of work. Three examples illustrate the range.

Derivation-heavy assignment

A graduate student in applied mathematics is given a problem set involving a stability proof. They use AI to check the structure of one step in the proof. The disclosure at the end of the problem set reads:

For Problem 3, I used an AI tool to verify my approach to the linearization step.

1. *Asked: "In a Lyapunov stability proof for $\dot{x} = f(x)$ with f smooth, is it sufficient to show that the linearization at the equilibrium has eigenvalues with negative real parts to conclude local asymptotic stability?"*
2. *Returned: Confirmed yes, with a reference to the Hartman-Grobman theorem and a note about hyperbolic equilibria.*
3. *Accepted: The conclusion that linearization gives local asymptotic stability when all eigenvalues have negative real parts and the equilibrium is hyperbolic. I restated this conclusion in my proof using the Hartman-Grobman framing.*
4. *Rejected: The AI suggested I cite Khalil's textbook for Hartman-Grobman; I checked Khalil and found the result is in Chapter 4 but not under that name. I cited the result by content rather than by chapter.*
5. *Verified: I worked the linearization at the equilibrium of my problem (eigenvalues $-1, -2$) by hand and confirmed both have negative real parts. I confirmed hyperbolicity by checking that no eigenvalue lies on the imaginary axis.*

Code-heavy assignment

An undergraduate in a numerical-methods course is implementing a finite-difference solver for a heat equation. They use AI to draft the time-stepping loop. Their disclosure reads:

For the implementation in Part B, I used an AI tool to draft the time-stepping loop.

1. *Asked: "Write a Python function that performs one explicit Euler time step for the 1D heat equation $u_t = \alpha u_{xx}$ on a uniform grid with Dirichlet boundary conditions."*
2. *Returned: A function with the standard explicit Euler stencil $u_i^{n+1} = u_i^n + \alpha \Delta t / \Delta x^2 \cdot (u_{i+1}^n - 2u_i^n + u_{i-1}^n)$, with boundary conditions enforced after the update.*
3. *Accepted: The stencil and the structure of enforcing boundary conditions after the update. I rewrote the function in my own style, using explicit array indexing rather than the AI's vectorized form, because I find the indexing version easier to verify.*

4. *Rejected: The AI used the variable name* `alpha` *for the diffusion coefficient and* `u_new` *for the updated solution. I changed these to match the notation in our textbook.*
5. *Verified: I tested the implementation against the analytical solution for a sinusoidal initial condition with zero boundary conditions. The simulated decay rate matched the analytical decay rate* $\exp(-\alpha\pi^2 t)$ *to four decimal places. I also confirmed the CFL stability condition* $\alpha\Delta t/\Delta x^2 \leq 1/2$ *by running a simulation with a step size violating it; the simulation blew up as expected.*

Open research project

A senior writing an honors thesis uses AI for several distinct purposes during the project. Their thesis-end disclosure reads:

During the preparation of this thesis, I used AI tools at four points.

(a) *Literature reconnaissance:* I asked an AI tool to help me identify the major theoretical frameworks in the literature on [topic]. The AI listed five frameworks; I read primary sources for three of them and rejected the other two on the grounds that the AI's characterization did not match what I found in the sources.

(b) *Code prototyping:* I used an AI tool to draft the simulation code in Chapter 3. The detailed disclosure for this code follows the five-field template above and is included in the supplementary material as `ai_disclosure_chapter3.txt`.

(c) *Prose editing:* I used an AI tool to suggest edits to the introduction and discussion sections of the thesis. I accepted edits that improved clarity and rejected edits that softened claims my data supported. I did not use AI to draft prose from scratch; the prose in this thesis is my own.

(d) *Verification:* I used an AI tool to spot-check several of my derivations. In each case, the verification confirmed my work; in two cases, the AI flagged a step as suspicious that, on examination, turned out to be correct (the flag prompted me to add an explanatory sentence to the derivation, which I would otherwise have omitted).

No AI tool generated any data, theorems, or interpretations that appear in this thesis. All conclusions are my own.

The third example illustrates an important point: in extended work, you may need to disclose at multiple levels of granularity. A thesis-end disclosure summarizes the categories of AI use; specific assignments or chapters may need their own detailed disclosures. The principle is that the reader should be able to find, at the appropriate level of detail, what AI did and what you did.

6.4 When disclosure matters and when it doesn't

Not every AI interaction needs to be disclosed. A reasonable rule of thumb: *disclose AI use that contributes to the substance of your work, not AI use that is incidental.*

Examples of substantive use that should be disclosed:

- AI-drafted derivations, code, or analyses that ended up in your submission.
- AI-suggested citations or references that you used.
- AI-proposed interpretations of results that you adopted.
- AI-assisted proofreading that changed the substance of the work.

Examples of incidental use that typically does not require disclosure:

- Looking up the spelling of a technical term.
- Asking for the syntax of a programming-language construct.
- Translating a notation between conventions for your own understanding.
- Asking for definitions of unfamiliar terms in a paper you are reading.

The line between substantive and incidental is sometimes blurry. The right test is whether the reader of your work would, if they knew about the use, want to factor that information into their evaluation of the work. If yes, disclose. If no, the use is genuinely incidental.

When in doubt, disclose. A disclosure that turns out to be unnecessary is a small cost; a non-disclosure that turns out to be necessary is a large one. Consult your institution's policies for the formal requirement, and adopt a personal standard that errs on the side of transparency.

6.5 Disclosure as a habit, not a chore

The hardest part of disclosure is making it routine. A student who disciplines themselves to write a five-field disclosure at the end of every assignment has built a habit that protects them in every context—academic, professional, and personal. A student who treats disclosure as a chore to be performed when forced will, at some point, fail to disclose something they should have, with consequences that are bounded only by how visible the failure becomes.

The disclosure template in Appendix B fits on one page. Print it, put it next to your desk, and use it. After a semester, the five fields will come naturally. After a year, you will not be able to imagine submitting work without them.

Instructors designing assessments that require disclosure should also see Chapter 12, which addresses documentation requirements, integrity boundaries, and the operational details of running a course where disclosure is part of the grade.

Chapter 7

Working with AI on Different Tasks

The previous chapters laid out principles. This chapter addresses the situations students actually face. Five common tasks, with concrete advice on how to use AI well in each.

The principles from earlier chapters—use AI to learn rather than to get answers; verify everything; respect the cognitive science; disclose substantive use—are not equally easy to apply across tasks. Studying for an exam invites different AI use than working a problem set, and writing a paper raises different issues than reading one. This chapter walks through five common tasks and the specific habits each rewards.

The five tasks below cover most of what students actually do during a semester: studying for exams, working problem sets, reading difficult papers, writing papers and reports, and conducting original research. The advice in each section assumes you have read Chapters 1 through 6; it does not repeat the foundational arguments but applies them concretely.

7.1 Studying for an exam

Exam preparation is where the difference between AI use that helps and AI use that hurts is largest. The exam is, by definition, a setting where AI is unavailable. Study habits that build memory accessible without AI pay off; study habits that build memory accessible only with AI prompts produce the worst possible exam outcome—the feeling that you knew the material, with no ability to demonstrate it.

What works: Use AI for self-testing. Have it generate questions about the material; close the AI tab; write your answers without consulting anything; only then ask AI to evaluate. Repeat across the topics you expect to be tested on. The retrieval practice (Chapter 5) is what builds exam-accessible memory.

What works: Use AI to identify your weakest topics. Ask the AI to generate a 30-minute mock exam covering the course; take it under timed conditions; identify the topics you got wrong. Spend the next study session on those topics, again with self-testing rather than re-reading.

What works: Use AI to surface cross-topic connections you might miss. After you have studied each topic individually, ask the AI to pose a question that requires combining two or three topics. The combined question reveals whether you can apply the material flexibly or only in the form you studied it.

What does not work: Using AI to summarize chapters or rewrite your notes in cleaner form. The clean summary feels productive and is mostly a re-reading exercise. You will recognize the material when the AI presents it back to you and fail to recall it on the exam. Do not confuse comprehension with learning.

What does not work: Asking AI to explain everything in three different ways until something clicks. The third explanation always clicks, in the moment, regardless of whether it is clearer than the first. This is the illusion of fluency (Chapter 5) at work; you are training yourself to recognize explanations rather than to produce them.

Exam-prep schedule

A schedule that uses AI well: 60% of study time on AI-generated self-test problems you answer without AI; 25% on working past exam problems and homework problems you have not done before;

10% on diagnostic AI sessions identifying weak topics; 5% on AI-generated explanations of specific stuck points. The ratios matter: a study schedule that flips them produces the illusion of preparedness without the substance.

7.2 Working a problem set

Problem sets are where AI temptation is strongest, and where the cost of giving in is highest in the medium term. The problem-set worth doing is one that develops a skill the exam will test; AI-completed problem sets develop the skill of submitting AI-completed problem sets, which the exam will not test.

What works: Engage with each problem yourself before consulting AI for anything. The 15-minute rule from Chapter 5 applies: spend at least 15 minutes attempting each problem before any AI consultation. The struggle, even unsuccessful, builds the scaffolding that makes the eventual solution land.

What works: If you get stuck, ask the AI for a hint, not a solution. "What technique applies to this kind of problem?" is a hint. "What concept do I need to understand before I can solve this?" is a hint. "Solve this problem" is not a hint. The hint preserves your role as the problem-solver; the solution displaces you from it.

What works: After you have solved a problem unaided, ask the AI to evaluate your solution. Did you make any errors? Did you use the most efficient approach? Were there steps you took for granted that needed justification? The evaluation is more useful than the solution because it engages with what you actually did.

What does not work: Pasting the problem into AI, copying the solution, and submitting it. Even if you understand the AI's solution well enough to defend it, you have not practiced the underlying skill. The next problem of the same type will be just as hard.

What does not work: Using AI to "check" a problem you have not actually attempted. Submitting AI's solution as if it were your own check is academic dishonesty and is also pedagogical self-sabotage; you have neither attempted the problem nor learned from the AI's solution.

Watch out

The hardest case in problem-set work is the problem you genuinely cannot solve even with effort. The temptation is to consult AI for a complete solution and "learn from it." The honest path is to (a) document where you got stuck, (b) ask AI a question targeted at that specific step, (c) work the problem to completion using AI's response on only that step, and (d) disclose the AI use per Chapter 6. The honest path is harder; it also produces actual learning.

7.3 Reading a difficult paper

Reading a research paper outside your immediate area of expertise is a task where AI assistance is unambiguously valuable, used well. The paper assumes background you may not have; the AI can fill the background gaps quickly.

What works: Use AI to clarify notation as you encounter it. Papers use notation that varies across subfields; asking the AI "what does $\mathcal{H}^k(X)$ mean in algebraic topology?" or "what does $\hat{\sigma}^2$ typically denote in time-series analysis?" takes seconds and lets you keep reading.

What works: Use AI to generate alternative explanations of a concept the paper introduces but does not fully explain. "Explain Itô's lemma in three different ways" produces the framing that lands; you then return to the paper with that framing in mind.

What works: Use AI to identify what the paper assumes you already know. "This paper assumes familiarity with [field]; what concepts from that field will I need?" produces a checklist; you then assess

which items you know and which need review.

What works: Use AI to pose comprehension questions about a section you have just read. Closing the AI window, answering the questions in writing, then returning to verify catches gaps you might have missed.

What does not work: Asking AI to summarize the paper before you read it. The summary will be partial, possibly wrong on technical points, and will give you a false sense of understanding the paper that the actual reading does not need to overcome. Read first; summarize second; compare your summary to AI's summary as a comprehension check, not as a substitute.

What does not work: Asking AI to "explain" a paper's main result if the AI lacks access to the paper. The AI will produce a plausible explanation of what the result probably says, which may not match what the paper actually says. The hallucination risk is high; the verification cost is also high (you have to read the paper anyway). Skip the AI step and read the paper.

7.4 Writing a paper or report

Writing is a task where the boundary between substantive and incidental AI use (Chapter 6) is most consequential. AI can help with prose at every level, from word choice to structural editing, and the line between "editing assistance" and "co-authorship" is blurry.

What works: Use AI to suggest edits to prose you have already drafted. "Here is a paragraph; suggest edits that improve clarity without changing meaning" is a clean prompt that respects your authorship. You evaluate each suggested edit and accept or reject it.

What works: Use AI to identify weaknesses in your argument. "What is the strongest objection a careful reader would raise to this paragraph?" produces feedback you can address. You do the addressing.

What works: Use AI to check citation formatting, equation typesetting, or other mechanical aspects of the manuscript. These are incidental uses (Chapter 6) and typically do not require disclosure.

What works: Use AI to draft *outlines* for sections, then write the actual prose yourself. The outline gives structural scaffolding; the prose you write yourself is yours.

What does not work: Using AI to draft prose from scratch and then "revising" it. The revision will be light because AI prose is fluent; the resulting paragraph will be more AI's voice than yours, the underlying argument will be AI's, and the disclosure burden (Chapter 6) becomes substantial. Better to write the paragraph yourself, even slowly, and use AI only for editing.

What does not work: Asking AI to write a section in the style of a specific author or paper. The pastiche risk is real (you may produce text that closely tracks an existing source) and the substantive contribution is none.

Substantive vs. incidental in writing

A graduate student is drafting the introduction to a paper. Compare two AI-use patterns:

Pattern A (substantive, requires disclosure): The student asks AI to write the introduction's opening paragraph framing the problem; AI returns a paragraph; the student edits it lightly and submits. The opening framing is AI's, not the student's. Disclosure is needed.

Pattern B (incidental, no disclosure needed): The student writes the introduction themselves. They ask AI to suggest more precise word choices for three terms they were unsure about. They accept one suggestion and reject two. The argument and structure are theirs; the AI helped at the level of vocabulary. No disclosure needed.

The two patterns produce different outputs and different obligations. The student in Pattern B has done the substantive work of intellectual contribution; the student in Pattern A has not.

7.5 Conducting original research

Original research is where AI use becomes most consequential and where the disclosure norms are evolving fastest. The advice in this section is more tentative than in earlier sections; the field is still working out what counts as appropriate AI use in research, and the answers will look different in five years than they do today.

What works: Use AI for literature reconnaissance. "What are the main theoretical frameworks in the literature on X?" produces a starting list; you then read primary sources for each framework and evaluate whether the AI's characterization is accurate. The AI accelerates the surveying; the reading remains your responsibility.

What works: Use AI for code prototyping (with the verification habits from Chapter 4). Research code that runs but does the wrong thing has caused multiple high-profile retractions; verification is not optional.

What works: Use AI to spot-check derivations and proofs. The AI will sometimes flag a step as suspicious that turns out to be correct; the flag prompts you to write a clearer version. The AI will sometimes endorse a step that is actually wrong; the endorsement is unreliable. Use AI flags as prompts for your own verification, not as authoritative judgments.

What works: Use AI to translate between subfields' notations and conventions when reading interdisciplinary literature. This is the "clarifying notation" use mode (Chapter 2); it accelerates reading without contributing substantively.

What does not work: Using AI to generate research hypotheses or theorems. AI-generated hypotheses are plausible-sounding combinations of known results; they rarely advance understanding because they reflect what is already in the AI's training data, not what is genuinely novel. Hypotheses worth investigating come from your engagement with the literature and your sense of what is missing, not from AI.

What does not work: Using AI to generate data, results, or experimental conclusions. This is research misconduct in any framing; the issue is well-settled in publication ethics even where formal policy is still developing.

What requires careful judgment: Using AI to draft research prose. The norm in 2026 is that AI-assisted writing must be disclosed for substantive use; the threshold of "substantive" is debated. When in doubt, disclose. The cost of over-disclosing is small; the cost of under-disclosing can be substantial.

Watch out

Research-grade AI use must be auditable. Keep records of your prompts and AI responses, especially for code and derivations that end up in published work. The records protect you if questions arise later about what AI did and what you did. "I don't remember exactly how the AI helped" is a bad answer to a question from a journal editor or a thesis committee; the records make a better answer possible.

Chapter 8

Self-Assessment and Common Traps

The previous chapters describe what to do. This chapter describes how to tell whether you are doing it. Four self-tests, four named traps, one closing argument.

The hardest part of using AI well is knowing when you are not. The illusion of fluency (Chapter 5) makes ineffective study feel productive; the convenience of AI-completed work makes substitute-for-thinking use feel like efficiency. Without explicit self-assessment, the feedback loop that would let you correct course is muted or absent.

This chapter offers four practical self-tests and four named traps that students consistently fall into. Both sets are most useful when revisited periodically rather than read once and forgotten; the tests catch shifts in your habits before the shifts produce visible damage.

8.1 Four self-tests

The retrieval test. At the end of any study session that involved AI, close every reference (textbook, notes, AI tool) and write down what you learned, in your own words, without consulting anything. Compare what you wrote to what the session was supposed to cover. The gap between the two is the gap between what you absorbed and what you retained. If the gap is large, the session built the illusion of fluency rather than durable memory; the next session should use more retrieval-practice mode and less AI-explanation mode.

The teaching test. Pick a concept you have just studied. Without consulting any reference, explain the concept to a peer who has not seen it—or, if no peer is available, explain it aloud to yourself or in writing as if to a peer. The teaching test is harder than the retrieval test because it requires you to organize the material for someone else; passing it indicates a level of understanding that goes beyond surface familiarity. Failing it indicates that you have absorbed the material but not yet structured it. Both outcomes are useful; the failure tells you what to work on next.

The variation test. Take a problem you have just solved (with or without AI assistance) and modify it slightly: change a parameter, swap an assumption, alter the boundary condition. Solve the modified problem without further AI consultation. If you can, you have understood the original problem well enough to apply the underlying technique. If you cannot, your engagement with the original problem was at the level of pattern-matching to the AI's solution rather than at the level of the underlying concept.

The honest-effort test. Before you consulted AI on a problem, did you genuinely try to solve it yourself? The 15-minute rule from Chapter 5 is the operational standard. The honest-effort test catches the most insidious failure mode: the student who consults AI "just to see how to start" and then never returns to genuine effort. The AI's start displaces the student's start; the student's eventual solution is a continuation of the AI's reasoning rather than their own.

Try this

Run all four tests at the end of each week of a course. Note which tests you passed and which you failed. After three weeks, look for patterns. If the retrieval test consistently fails, you are over-using AI explanations; shift toward self-testing. If the variation test consistently fails, you are pattern-matching rather than understanding; spend more time with the conceptual foundations. If the honest-effort test consistently fails, you are consulting AI too early; impose a longer pre-AI

struggle window. The patterns reveal which habit needs adjustment.

8.2 Four common traps

Beyond the failure modes catalogued in Chapter 3, four traps recur in students' AI-use habits. The traps are about *your* use of AI rather than about AI's behavior. Recognizing the trap by name makes it easier to avoid.

Plausibility bias. The trap of accepting AI output because it sounds confident, regardless of whether it is correct. AI tools produce fluent prose by design; fluency is not evidence of accuracy. Students who fall into plausibility bias rate AI-generated work as more correct than identical work labeled as a peer's. The corrective is the verification habit (Chapter 4): *plausibility is what you check after, not what you trust before.*

Complexity inflation. The trap of believing that an AI's elaborate solution is better than your simpler one. AI tools sometimes produce solutions that are more sophisticated than the problem requires—using machinery from advanced topics to solve a problem that yields to elementary methods. The complexity feels impressive and the student accepts it; the simpler approach the student would have produced is dismissed as inadequate. The corrective: *simpler is better when both work.* If your method gives the same answer as the AI's method, your method is preferable, especially on problems whose pedagogical intent is to teach the simpler approach.

The conversation trap. The trap of asking an AI to verify its own previous output. "Are you sure?" is the canonical conversation-trap query. The AI's response will be plausible and will not catch errors in its own reasoning, because the AI does not have an independent verification mechanism; it is generating new fluent prose, not running a check. The corrective: *verify by independent means, not by re-asking the same AI.* Compute by hand; consult a different source; ask a peer; check units. Do not ask the AI to grade itself.

The velocity trap. The trap of mistaking faster work for deeper work. AI accelerates production; it does not accelerate understanding by the same factor. A student who completes a semester's homework in half the usual time has not learned twice as much; they have learned a fraction as much, with the rest of the time available for other things (which may be valuable, but is not what the homework was supposed to produce). The corrective: *evaluate study sessions by what they built, not by what they completed.* Ten problems solved by AI is not ten problems' worth of skill development; it is one problem's worth of typing.

The traps in combination

A graduate student is preparing for a qualifying exam. They use AI to work through practice problems, accepting the AI's solutions because they look right (plausibility bias) and using sophisticated techniques the AI suggests rather than the simpler textbook approaches (complexity inflation). When uncertain about a step, they ask the AI to confirm (the conversation trap). They complete forty practice problems in a week (the velocity trap). On the actual qualifying exam, none of the four traps protects them: the problems require understanding they did not build, in approaches the textbook taught and they did not practice, with no AI to consult. The exam goes badly.

The same student, applying the four self-tests across the same week, would have completed perhaps fifteen problems—worked them themselves, verified them independently, varied them to test understanding, and corrected for honest effort. The fifteen-problem week produces exam preparation. The forty-problem week does not.

8.3 Closing argument: the long horizon

The advice in this guide is calibrated to a long horizon. In any single week, the gap between effective AI use and ineffective AI use is small; both produce completed work, both feel productive, both may earn similar grades. The differences accumulate slowly.

Over a semester, the student who uses AI well builds skills the student who uses AI poorly does not. Over a degree, the gap is the difference between a student who can do the work and a student who can submit work. Over a career, the gap is the difference between a professional who happens to use AI tools and a professional who depends on them and is hollow without them.

The long horizon is what makes this guide worth your attention now. The habits described here—the verification check, the disclosure routine, the retrieval-practice schedule, the self-tests, the named traps—are small enough to adopt and consistent enough to maintain. None requires heroic discipline; all reward sustained attention.

Start this week. Pick one habit from this guide and apply it to one course you are taking. After a week, evaluate. Add a second habit. After a semester, the habits will be invisible to you but visible to anyone watching—the friends who notice you understand the material, the instructors who notice you ask better questions, the future you who finds the work easier because the foundation is solid.

The AI will be different in five years. The habits will not. Build them now.

Readers who also teach—instructors, teaching assistants, or students who tutor—will find Part II of this guide useful as well. The student habits described in Part I are not separable from the course environment that supports them; an instructor who teaches one course in a way that rewards substitute-for-thinking AI use undermines the habits Part I is trying to build, regardless of what other courses encourage. Part II addresses how to design courses that actually develop the habits Part I describes.

Part II

For Instructors

Chapter 9

Teaching with AI: The Pedagogical Stance

The first task in teaching with AI is not designing assignments. It is taking a position on what students are supposed to be learning.

Part I of this guide is written for students. Part II is written for the instructors who teach them. The two audiences need different things, but they share one assumption: that AI tools are now a permanent feature of technical study, and that pretending otherwise—through prohibitions that cannot be enforced or policies that cannot be sustained—fails students more than it serves them. Instructors who have not yet read Part I will find it useful as preparation; the principles of effective student AI use developed in Chapters 1–8 are what Part II's course-design recommendations are calibrated to support.

This chapter sets out the pedagogical stance the rest of Part II rests on. The stance has three components: a clear position on what the course is teaching, a set of principles that distinguishes responsible AI use from substitute-for-thinking use, and an honest reckoning with what AI changes about the instructor's role. The chapters that follow build on this foundation: course design (Chapter 10), assessment instruments (Chapter 11), and disclosure norms and academic integrity (Chapter 12).

9.1 What the course is for

Before designing AI-aware assignments, decide what your course is supposed to produce in your students. The answer determines everything else.

A graduate course in mathematics produces a student who can do mathematics. The mathematics they do may now be supported by AI tools, but the doing remains theirs. A student who completes the course having outsourced the mathematics to AI has not completed the course; they have completed a different course, one that produces query-composers rather than mathematicians.

The same logic applies to any technical course. A statistics course produces a student who can analyze data; a programming course produces a student who can write code; a physics course produces a student who can reason from principles. AI tools can support each of these, but the underlying capacity must develop in the student. If it does not, the course has failed regardless of what grades it issues.

This is not a controversial position when stated explicitly. It becomes controversial only in the implementation, when an instructor must distinguish between AI use that supports the underlying capacity and AI use that displaces it. The distinction is the central design problem of AI-aware course development, and the rest of Part II is about how to navigate it.

9.2 Five principles that organize the rest of Part II

Five principles organize the course-design and assessment chapters that follow. They are stated here in summary form; the chapters elaborate.

Mathematical depth comes first. Whatever the role AI plays in your course, students must demonstrate genuine conceptual understanding independent of AI. A student who cannot do the mathematics without AI has not yet learned the mathematics. This principle has a corollary in assessment: at least some of your assessments must be AI-free, and a minimum threshold of AI-free competence must be required to pass.

AI literacy is a learnable skill. Students do not arrive in your course knowing how to use AI productively for mathematical work. The skill includes knowing what to ask AI, recognizing when AI output is wrong, integrating AI suggestions into rigorous work, and disclosing AI use clearly. Treat these as competencies your course teaches, not as background students should already have.

Epistemic responsibility is non-negotiable. Students must verify, critique, and transparently document AI contributions. This is the third axis of competence (alongside mathematical depth and AI literacy), and it is the one most easily neglected in course design. The disclosure habit is built through repeated practice and graded artifacts; it does not develop on its own.

Process matters as much as product. What students did to produce a piece of work is now a primary object of assessment, not a secondary one. The reasoning shown, the verification performed, the prompts used, and the rejected suggestions are all evidence of competence. A grading scheme that evaluates only final answers cannot distinguish a student who genuinely solved a problem from one who copied AI's solution.

Expectations must be explicit, consistent, and fair. Students cannot navigate AI policy that varies arbitrarily across courses or that is unwritten until violated. The instructor's job includes stating clearly, in writing, what AI use is permitted on each assessment type, what disclosure is required, and how the work will be graded. The clarity is itself part of the educational contribution.

These five principles are mutually reinforcing. A course that takes mathematical depth seriously will design AI-free assessments. A course that takes AI literacy seriously will design assessments that build it. A course that takes epistemic responsibility seriously will require disclosure. A course that takes process seriously will grade on visible reasoning. A course that takes equity seriously will state expectations clearly. Together, the five principles produce a course that teaches students to be the kind of mathematician (or scientist, or engineer) who uses AI well—which is the central educational goal of teaching in the AI era.

9.3 What AI does not change about the instructor's role

It is easy, in conversations about AI in education, to overstate what has changed. Several things have not changed and are unlikely to change soon.

The instructor still needs to know the subject deeply. AI does not replace expertise; it raises the value of expertise, because the instructor is now the only reliable verifier of student work, the only authority on what counts as a correct answer in the course's context, and the only person who can distinguish substantive AI use from substitute-for-thinking AI use. An instructor who delegates these judgments to AI tools has abdicated the role.

The instructor still needs to design assignments that test what they want students to learn. AI tools may make some traditional assignment types unreliable as assessments, but the principle of designing assessments to match learning goals is unchanged. Chapter 10 develops this in detail.

The instructor still needs to provide feedback that engages with student reasoning. AI can grade for surface correctness; AI cannot tell a student that their proof is technically correct but conceptually backwards, or that their statistical model is fitting noise, or that their code works but reflects a misunderstanding of the underlying physics. The pedagogical relationship between instructor and student remains a human relationship.

The instructor still bears responsibility for the integrity of the educational experience. If students leave your course unable to do the mathematics, you have failed them, regardless of how well they used AI tools. The accountability is yours.

9.4 What AI does change

Several things have changed and require active response.

Take-home work alone can no longer be trusted as a measure of mathematical competence. A traditional problem set, completed on the student's own time without restrictions, now measures a combination of the student's mathematical skill and their AI use; the two cannot be disentangled from the submitted artifact alone. This does not mean take-home work should be abolished—it remains valuable for developing skills and building habits—but it does mean the take-home component cannot stand alone as evidence of competence.

The set of skills you should be teaching has expanded. Verification, disclosure, prompt construction, and AI-output critique are now mathematical skills in the same sense that proof construction and computation are mathematical skills. They take instructional time and require explicit practice. Squeezing them into the course is not optional.

Assessment has become a multi-instrument enterprise. Single-instrument assessment (the final exam, the term paper) is no longer adequate; the instruments must work in combination, with each addressing a different competency under different tool conditions. Chapter 11 develops the assessment-instrument framework.

The integrity boundary has moved. The line between "the student's work" and "not the student's work" was once relatively clear: it was the work the student physically produced. The line is now blurrier, because student-AI collaboration covers a continuum from incidental tool use to extensive co-authorship. Drawing the line clearly, and communicating it clearly to students, is now part of the instructor's job.

These changes are real, and addressing them takes effort. The chapters that follow lay out concrete approaches: course-design choices that work with rather than against AI use (Chapter 10), assessment instruments that probe each competency under appropriate tool conditions (Chapter 11), and disclosure and integrity policies that can be sustained over time (Chapter 12). The pedagogical stance set out in this chapter is what those approaches are designed to serve.

Chapter 10

Course Design: Three Tool Regimes

The central organizing idea in AI-aware course design is that different assessments live in different tool regimes. Three regimes do most of the work.

A course that prohibits AI on every assignment is not viable in 2026; the prohibition cannot be enforced, and the resulting policy is unfair to honest students. A course that permits AI on every assignment cannot reliably measure mathematical competence; the take-home submissions cannot be distinguished from AI submissions. The way forward is neither prohibition nor permission, but a deliberate mix: different assignments operate under different tool conditions, and each tool condition addresses a different aspect of competence.

This chapter develops the three-regime model: *no-AI* assessments, *bounded-AI* assessments, and *open-AI* assessments. Each has a role; each has design constraints; each contributes evidence about a different facet of student competence. The chapter closes with weighting recommendations: how much of the course's grade should come from each regime.

10.1 No-AI assessments

The no-AI regime is the closest to traditional assessment. Students complete the assessment without AI tools, in conditions where the absence of tools can be enforced—typically, in-class, on paper or on a controlled laptop, with the network disabled or absent.

The pedagogical purpose of no-AI assessment is to establish baseline mathematical competence. Whatever students can do in this regime is what they can do without AI. The result is a clean measurement, uncontaminated by tool-use questions. This is the regime in which mastery of definitions, the ability to construct proofs, the capacity for fast on-the-spot problem-solving, and the holding of mathematical concepts in active memory are all measured.

Typical no-AI instruments include in-class written exams, short proof-and-concept tasks under timed conditions, and oral examinations at the blackboard. Each has different strengths. Written exams scale well and produce a paper trail; short tasks are quick to deploy and grade; oral exams are the most informative per minute of instructor time and the hardest to game.

Three design principles for no-AI assessments:

Keep the time pressure realistic. Students under extreme time pressure produce work that does not reflect their understanding; students with too much time begin to wonder what to do with the extra. The right amount of time depends on the assessment, but err on the side of slightly more time than seems necessary; the goal is to measure understanding, not speed.

Test understanding, not memorization. A no-AI exam that requires students to recall theorem statements verbatim measures memorization. An exam that asks students to apply a theorem to a problem they have not seen, or to identify what changes when a hypothesis is removed, measures understanding. The second is more durable as an assessment in the AI era because it cannot be prepared for by memorization alone.

Reserve the no-AI regime for what it does best. If a question can be answered just as well with AI assistance, putting it in a no-AI exam wastes the regime's distinctive value. The no-AI regime is for the questions where the absence of AI is part of the measurement.

A no-AI exam question

Weak (memorization-dependent): "State and prove the spectral theorem for symmetric matrices."

Strong (understanding-dependent): "A student claims that every diagonalizable matrix is symmetric. The claim is false. Give a 2×2 counterexample, and identify the precise hypothesis the student has confused."

The strong version cannot be answered by recall; it requires the student to know what hypotheses the spectral theorem actually uses, and to construct a counterexample on the spot. This is what no-AI assessment is for.

10.2 Bounded-AI assessments

The bounded-AI regime occupies the middle ground. Students use AI tools, but under specific constraints—a single tool, a fixed time window, mandatory disclosure, particular allowed and disallowed uses. The regime captures something the no-AI regime cannot: a student's ability to use AI well.

The pedagogical purpose of bounded-AI assessment is to measure AI literacy under controlled conditions. The constraints make the measurement clean: every student has the same tools and time, so observed differences in performance reflect differences in AI-use skill rather than differences in resources. The regime also produces evidence about a student's verification and critique habits, because the constraints (time, single tool, mandatory disclosure) push students to use AI deliberately rather than reflexively.

Typical bounded-AI instruments include AI-assisted take-home exams with strict time limits, AI critique and error-detection tasks (the student is given an AI-generated proof or analysis with subtle errors and must locate them), and human-AI comparative problem-solving tasks (the student solves a problem both with and without AI assistance and compares the experiences).

Three design principles for bounded-AI assessments:

Specify the bound clearly. "Use AI for at most 30 minutes; record every prompt and response; submit the prompts with your work." Vague bounds invite vague compliance. The constraint should be specific enough that a student who follows it produces a clearly auditable record and a student who violates it produces a record that visibly fails the bound.

Reward the verification, not the answer. A bounded-AI assignment that grades primarily on the final answer measures something close to the open-AI regime. A bounded-AI assignment that grades primarily on the prompt log, the verification work, and the student's articulation of which AI suggestions were accepted and rejected measures the AI-literacy competency the regime is designed for.

Use bounded-AI for the AI-distinctive skills. The skills bounded-AI is best at measuring—prompt design, error detection in AI output, integration of AI suggestions with rigorous reasoning—are not the skills no-AI assessment can measure. Use the regime for what it does best.

A bounded-AI exam task

Task: You are given an AI-generated proof of the following claim: [claim]. The proof contains at least one subtle error. Identify the first invalid step, explain why it is invalid, and produce a corrected proof. You may use the AI to help you analyze the original proof, but you must verify any AI-suggested correction independently. Submit your final corrected proof, your verification work, and a short disclosure (10-15 lines) describing how you used AI.

This task tests three things at once: the student's ability to read a proof critically (D2 in the GMIP framework), their ability to detect AI errors (D7), and their disclosure discipline (D8). It cannot be gamed by AI alone, because the AI generated the original (flawed) proof; the student's contribution

is the detection and repair.

10.3 Open-AI assessments

The open-AI regime permits unrestricted AI use, with the requirement that the student document the use and verify the work. This is the regime closest to actual professional practice, where mathematicians and scientists routinely use AI tools without time limits or single-tool constraints.

The pedagogical purpose of open-AI assessment is to measure the student's ability to do professional-quality work in realistic conditions. The work is judged on its mathematical correctness, its quality of exposition, the rigor of its verification, and the transparency of its disclosure. The student is treated as a professional, with the responsibilities professionals carry.

Typical open-AI instruments include take-home problem sets with unrestricted AI use and required disclosure, AI-assisted mini research projects, conjecture-laboratory tasks (computational exploration leading to conjecture, proof attempt, and failure analysis), and project-based summative assessments.

Three design principles for open-AI assessments:

Require process artifacts as part of the deliverable. A plan written before tool use; a tool log of prompts, outputs, and acceptance/rejection decisions; a verification checklist showing what was checked and how; and a brief disclosure statement. These artifacts are not bureaucratic overhead; they are the primary evidence of the AI-literacy and epistemic-responsibility competencies. Grade them.

Two-layer grading. Grade each open-AI submission in two layers: (i) mathematical correctness and exposition, (ii) process integrity and verification. The two-layer scheme prevents a polished AI-generated answer from earning full credit when the student's process was undisciplined, and it prevents a partial answer with strong process from being penalized as if it were a careless answer.

Use open-AI for tasks that benefit from extended engagement. The open-AI regime is the right home for tasks that require sustained thought, computational exploration, careful writing, or integration of multiple sources—the tasks that reward time invested. Routine problem sets do not need the open-AI regime; reserve it for assignments where the engagement matters.

An open-AI assignment

Task: Investigate the following conjecture: [conjecture]. Your investigation should include (a) a brief plan written before any tool use, (b) computational exploration including at least three numerical experiments, (c) an attempted proof or counterexample, (d) a failure analysis if the attempt fails, and (e) a discussion of what you would do next given more time. You may use any AI tools. Submit your written report (8–12 pages), your computational notebooks, and a complete tool log with disclosure statement.

This is what professional-quality mathematical investigation looks like in 2026. The assignment treats students as the mathematicians they are becoming, and the rubric grades them on the qualities that distinguish responsible practice from undisciplined tool use.

10.4 Recommended weighting

How should the three regimes be weighted in the course's overall grade? The answer depends on the level of the course and the program's goals, but a useful default for graduate technical courses is:

- *Mathematical Understanding (M)*: 40–50% of the course grade, drawn from no-AI assessments.
- *AI-Augmented Skill (A)*: 30–40%, drawn from bounded-AI and open-AI assessments.

- *Epistemic and Ethical Responsibility (E)*: 15–25%, drawn from disclosure quality, verification artifacts, and process documentation across both AI regimes.

Within these weights, two principles are non-negotiable:

A student must meet a minimum threshold in M to pass. AI proficiency cannot compensate for weak mathematical foundations. A student with strong AI skills but weak mathematics has not yet completed the course. The minimum-M-threshold rule enforces this.

The weighting should be stated explicitly in the syllabus. Students cannot prepare for a course whose assessment weights are unwritten. Explicit weighting respects student autonomy and supports the equity-and-clarity principle from Chapter 9.

For undergraduate courses, the weights typically shift toward higher M (50–60%) and lower A (20–30%), reflecting the foundational role of undergraduate technical education. For research-track graduate seminars, the weights shift toward higher A (40–50%) reflecting the closer match to professional practice. The specific numbers matter less than the principle: each axis of competence is weighted, the M minimum is enforced, and students know the weights in advance. Appendix D provides three sample syllabus statements calibrated to different course types (foundational graduate, research-track graduate seminar, advanced undergraduate) showing how these weights and the AI-use policy can be communicated to students.

Chapter 11

Assessment Instruments

The previous chapter described three tool regimes. This chapter describes the specific assessment instruments that operate within them, the competencies each instrument measures, and how to grade them.

A course's assessment plan is more than a schedule of exams. It is a set of instruments, each designed to measure particular competencies under particular conditions, working together to produce a defensible picture of what each student can do. This chapter develops the assessment-instrument toolkit, organized around the eight competency domains that graduate mathematical work draws on.

The eight domains are introduced first; the instruments follow, with each instrument's strengths, limitations, and grading criteria specified. The chapter closes with a short discussion of grading rubrics and the calibration practices that keep grades comparable across instructors.

11.1 Eight competency domains

A graduate-level technical course draws on a wide range of student competencies. Eight domains cover most of what your assessment plan needs to address. The domains are calibrated to the cognitive-science evidence reviewed in Chapter 5: each is a competency that retrieval practice and productive failure can build, that fluency-illusion can mask, and that cognitive offloading can erode. Designing assessments around these domains is what makes the cognitive-science framing operational at the course level.

D1. Conceptual understanding and definitions. Students can state definitions precisely, explain motivations, give nontrivial examples and counterexamples, and identify what changes when hypotheses change. Evidence: definition recall and reformulation in plain language; minimal counterexample construction under relaxed assumptions; dependency tracing across a derivation.

D2. Proof construction and proof repair. Students can write complete proofs, choose strategies, and repair broken arguments. They can certify whether a tool-produced proof is correct. Evidence: independent proof outline before tool use; annotated proof audit identifying gaps and citing lemmas; error-localization tasks finding the first invalid inference.

D3. Problem solving and technique selection. Students can decide what to try next, not just execute a known routine. They can explain why a technique is appropriate and when it fails. Evidence: strategy memos with alternatives and failure modes; timed problems with branching choices; post-solution reflection on what generalizes and what does not.

D4. Mathematical communication. Students can write and speak with precision and audience awareness, honest about uncertainty. Evidence: short expository notes with a guiding narrative; board proofs with clean structure and signposting; useful peer review of others' work.

D5. Modeling and interpretation (when relevant). Students can translate between assumptions, mathematical form, and interpretation. They can state what a model proves, what it suggests, and what it cannot justify. Evidence: assumption-to-consequence maps; sensitivity and robustness reasoning; clean separation of theorem, computation, and interpretation.

D6. Computational mathematics and experimentation. Students can use computation to explore, test conjectures, and validate steps. They can reason about numerical error and reproducibility. Evidence: reproducible notebooks; sanity checks, edge cases, alternative implementations; complexity and stability discussions at the right level.

D7. AI tool proficiency for mathematical work. Students can use AI tools to accelerate work while maintaining control over correctness. They prompt well, constrain outputs, and integrate tool output into rigorous work. Evidence: prompt-and-response logs with rationale; iterative refinement steering a tool toward a valid proof or counterexample; tool triangulation comparing results across methods.

D8. Research integrity, ethics, and professional judgment. Students disclose tool use honestly, respect attribution norms, and maintain rigorous standards. They avoid laundering tool output as personal work. Evidence: transparent disclosure statements; attribution and citation discipline; integrity under questioning, with a consistent story of how the work was produced.

Not every course needs to assess all eight. A first-semester graduate course might focus on D1–D4; a research-track seminar might emphasize D6–D8; a course on mathematical modeling might add D5. The choice of which domains to assess is a course-design decision that should be made explicitly and stated in the syllabus.

11.2 Four assessment instrument types

Four instrument types do most of the work in a well-designed assessment plan. Each operates primarily in one of the three tool regimes, addresses a particular subset of the eight competency domains, and rewards a specific kind of student effort.

In-class exams. The traditional instrument, run in two modes. *No-AI mode*: closed-network, no tools, focused on D1–D4. Short and frequent works better than long and infrequent; problems should be chosen so a correct solution depends on one or two key decisions, allowing reasoning to be graded rather than output volume. *Bounded-AI mode*: a controlled tool setting (one model, limited time, mandatory disclosure), focused on D7–D8 in addition to the core domains. Bounded-AI in-class exams are most useful when the bound is short (30–60 minutes total tool time within a longer exam) and the disclosure requirement is substantive.

Take-home work with strict process. Open-AI take-home assignments graded in two layers: mathematical correctness and exposition (D1–D6) plus process integrity and verification (D7–D8). The two-layer grading is what distinguishes this instrument from a traditional take-home; the process artifacts (plan-first section, tool log, verification checklist, disclosure statement) carry weight in the grade rather than being optional appendices. Strong default: take-home work counts as evidence of D7 (AI proficiency) and D8 (research integrity) rather than primarily as evidence of mathematical competence.

Oral examinations. The most future-proof instrument because it probes live understanding. A 30–60 minute session with two segments: (1) blackboard reasoning—definitions, short proofs, probing questions that test hypothesis-awareness; (2) audit-and-repair—the student is given a short argument (possibly tool-generated) containing subtle flaws and must repair it. Oral exams measure D1–D4 cleanly, and the audit-and-repair segment provides the most reliable evidence available of D7. They are also the assessment most resistant to AI substitution, because the AI cannot answer in real time on the student's behalf. Appendix E provides a working bank of question types organized into five categories (definitions and motivating examples; short proofs; hypothesis stress tests; audit-and-repair; synthesis), with sample questions in each that can be adapted to specific courses.

Research-style performance tasks. Once-per-term assignments that resemble real mathematical work: exploring a conjecture, building a small theory, producing a carefully written note. These tasks should reward disciplined verification, good exposition, and honest handling of uncertainty. Specific forms include: *conjecture laboratory* (computational exploration → conjecture → proof attempt → failure analysis); *counterexample clinic* (generate families of examples that stress-test a statement); *mini-survey* (synthesize a small literature slice with correct attribution and clean exposition). These tasks measure D5–D8 in particular, plus D1 and D4 along the way.

A typical course assessment plan combines all four instruments in proportion: in-class exams contribute 30–40% of the grade, take-home work 25–35%, oral examinations 15–25%, and research-style tasks 10–20%. The weights are not magic; they are starting points to adjust based on course level, class size, and program goals.

11.3 Grading rubrics

Each assessment is graded using a small set of common anchors. The anchors are stable across instruments and across courses, which makes grades comparable and feedback portable. Five anchors handle most cases:

Correctness. The mathematics is correct, complete, and appropriately justified. This is the foundational anchor; weakness here cannot be compensated for by strength elsewhere.

Reasoning visibility. The work shows the path, not just the destination. A correct answer with no reasoning visible is barely better than a missing answer; the path is what is being assessed in graduate work.

Verification discipline. The student checks claims and catches errors, including tool errors. This anchor is increasingly important as AI use becomes standard; it is the single best predictor of whether a student is using AI well or being used by it.

Communication. Structure, definitions, and reader guidance are strong. Mathematical communication is itself a competency (D4) and a contributor to the value of every other anchor; weakness here makes the rest harder to assess.

Integrity. Tool use is disclosed and consistent with course rules. This anchor is binary in extreme cases (failure to disclose required AI use is a failure regardless of how good the mathematics is) and graded in routine cases (incomplete disclosure carries a small penalty; thorough disclosure carries no bonus, because thorough disclosure is the expected baseline).

The specific rubric applied to each instrument should be visible to students before they take the instrument. A rubric written after the fact, even if accurate, breaks the equity-and-clarity principle from Chapter 9.

11.4 Calibration and consistency

For a multi-instrument assessment plan to produce defensible grades, instructors who teach the same course (or different sections of it) need to grade consistently. This does not mean grading identically; it means a stable interpretation of what "mastery" and "responsible practice" look like across the course.

A practical mechanism: each term, instructors grade a small sample of student work (5–10 pieces, drawn from across the course's assessments) using the shared anchors. They then compare grades and discuss discrepancies. The goal is not to converge on identical scores, but to develop a shared sense of what each anchor means in practice. After a few rounds of this calibration, grades across instructors become genuinely comparable; without it, they are at best loosely related.

For courses taught by a single instructor, the analogous practice is to grade a small sample of work twice—once at the start of grading, once at the end—and check whether the rubric was applied consistently. Drift over a stack of papers is normal and easy to correct once detected.

Chapter 12

Documentation, Disclosure, and Academic Integrity

The most challenging part of teaching with AI is not technical. It is operational: building a documentation and disclosure regime that students can sustain and that you can grade.

The academic-integrity infrastructure that worked before AI tools—honor codes, plagiarism detection, in-class exams—does not fully extend to a world where every student has access to a sophisticated mathematical assistant. New infrastructure is needed: documentation requirements that produce auditable records, disclosure norms that students can follow consistently, and integrity boundaries clear enough to enforce. This chapter develops the operational details.

12.1 What students must document

For any AI-integrated assignment, students should submit a documentation packet that includes the following items. Each item supports a different aspect of the assessment.

Tools used. The AI system, the version (when known), and the dates of the sessions. This is brief but essential; it is the entry point for anyone auditing the work.

Prompt evolution. The prompts the student gave the AI, in order, with brief notes on why each was chosen and how earlier responses shaped later prompts. The prompt log is the clearest evidence of D7 (AI proficiency); a student who shaped prompts carefully has demonstrated something a student who pasted the problem and accepted the first response has not.

AI-generated outputs. The AI's actual responses, in summary or in full as appropriate. Students sometimes resist providing AI outputs because they feel exposed; the response should be that the documentation is the record of professional practice, not a confession. The outputs make the verification artifacts auditable.

Independent verification and corrections. What the student checked, how they checked it, what they accepted, and what they corrected. This is the heart of the documentation packet because it is the evidence of D7–D8 working together: AI was used, errors were caught, the final work is correct.

Final mathematical reasoning. The work as the student submits it. This is the artifact graded for mathematical correctness; the rest of the packet supports the assessment of how the artifact was produced.

The documentation packet does not need to be elaborate. For a typical homework problem, two pages of documentation are enough; for a research-style task, the documentation might run to ten pages. The principle is that the packet should be detailed enough that a careful reader can reconstruct what happened, and no more detailed than that.

12.2 Minimal integrity boundaries

A few bright lines, enforced consistently, are more useful than a long list of rules with occasional exceptions. Four boundaries work well as defaults.

Submitted proofs and analyses must be auditable. A student may not submit AI output as a proof without an audit identifying why each step is valid. The audit is the student's contribution; without it, the submission is laundering.

Oral defense is required. If a student cannot explain an argument orally—in a brief follow-up conversation, at office hours, or during an oral exam—the argument does not count as theirs for purposes of the course. This boundary is stated up front and applied consistently.

All external sources must be disclosed. AI tools, peer collaborators, online resources, textbooks beyond the assigned ones—each is an external source. The disclosure standard does not vary by source type; the practice of disclosing is what matters.

Each assessment specifies allowed and disallowed uses. Students cannot follow rules that are unwritten. The syllabus, and each assignment's instructions, should state clearly what AI use is permitted and what is not.

Beyond these four, additional rules are course-specific. The four above are the minimum; they should be in every course that allows any AI use at all.

12.3 The integrity-under-questioning standard

The most reliable test of whether a student's submitted work is genuinely their own is whether they can defend it under questioning. The questioning does not need to be hostile or extensive; a few minutes of conversation is usually enough.

A student who produced the work themselves can typically: explain why they chose the approach they used; identify the trickiest step and articulate why it was tricky; describe one alternative approach they considered and rejected; respond coherently to a follow-up question that probes a specific intermediate claim. A student who outsourced the work cannot, in general, do these things consistently.

The integrity-under-questioning standard does not require formal oral exams (though oral exams are the cleanest implementation). It can be applied informally: a brief conversation at office hours about a homework problem, a follow-up question after a class presentation, a casual "can you walk me through this part?" during a project review. Students who know the standard exists, and who experience it occasionally, develop the habit of producing work they can defend.

12.4 What to do when integrity fails

Integrity failures will happen. The instructor's response shapes whether the failure becomes a teachable moment or an adversarial confrontation.

A useful default: when a student's work appears to have been produced primarily by AI without disclosure, schedule a follow-up conversation rather than issuing an immediate accusation. The conversation has two purposes. First, it gives the student an opportunity to demonstrate that they did the work themselves, which sometimes happens (the appearance can be misleading). Second, it makes the integrity-under-questioning standard real to the student in a way that abstract policy cannot.

If the conversation confirms that the student outsourced the work without disclosure, the response should be proportionate to the failure and the student's career stage. A first-semester graduate student who outsourced a homework problem benefits more from a serious conversation about disclosure expectations and a regrade-with-disclosure than from a sanction that ends the term. A senior graduate student who repeatedly submits AI work as their own on a final research project is in different territory.

The integrity-under-questioning standard also serves a preventive role: students who know they may be asked to defend their work tend to do work they can defend. The conversation does not need to happen often to have the effect; the possibility of it is what changes behavior.

12.5 The instructor's own disclosure

If you, as an instructor, use AI tools in producing course materials—generating problem variants, drafting feedback, sketching example solutions—disclose this to your students in the syllabus. The disclosure does not need to be elaborate; a brief statement that AI tools were used in some materials, with an indication of which kinds, is sufficient.

The reason for instructor disclosure is consistency. A course in which students are required to disclose AI use, taught by an instructor who uses AI without disclosure, sends a confused message about what the disclosure norm is for. Faculty disclosure aligns the norm; it also gives students an early-career model of professional disclosure practice in a low-stakes setting.

Two patterns work well. *Light disclosure*: "AI tools were used in preparing some problem statements and example solutions in this course; the underlying mathematics was verified by the instructor before posting." *Substantive disclosure*: when a particular assignment was generated with substantial AI involvement, note this on the assignment itself, with an indication of what the AI did and what the instructor verified.

The instructor's disclosure habit is, additionally, a chance to demonstrate the disclosure templates the course teaches. A syllabus that includes a faithful instance of a five-field disclosure (for some specific course material) is more pedagogically valuable than a syllabus that requires students to disclose without modeling what the disclosure looks like.

Appendix A

Generic Verification Checklist

One page. Print it. Keep it next to your desk.

For every piece of AI-generated technical content—a derivation, a code snippet, a literature summary, a worked example—ask the following questions before accepting it.

Mathematical and quantitative work

1. *Are the units consistent on both sides of every equation?*
   Write the units of every parameter and variable above its first appearance. Verify that every equation matches.

2. *Does the result reduce to a known special case under appropriate parameter limits?*
   A general result must reduce to known special cases. If it doesn't, the general expression is wrong.

3. *Does a single numerical instance, computed independently, agree with the AI's result?*
   Plug in specific numbers. Evaluate both sides. Compare. Disagreement reveals the error.

4. *Are all the assumptions stated in the question used somewhere in the derivation?*
   If an assumption is stated but unused, the derivation is suspect.

Citations and sources

5. *If a citation is given, can I retrieve the cited source and verify what it actually says?*
   Search for the paper in a real database. If you cannot find it, do not cite it.

Code and computation

6. *Can I read the code line by line and confirm each line implements what I intended?*
   Code that runs is not verified. Code that produces a known correct answer on a test case is verified for that case.

7. *Have I tested the code on at least one input where I know the right answer independently?*
   Hand-compute, consult a textbook, or use a different tool. Verify the code reproduces the known answer.

Prose and meaning

8. *If the AI rephrased a technical claim, do the quantifiers and hedges match the original?*
   "All" versus "most"; "proves" versus "suggests"; "in this study" versus "in general." Compare word by word.

Notation and conventions

9. *Is the notation internally consistent throughout the response?*
   Source conflation is invisible until you check. Pick a single convention and translate the AI's response into it.

A piece of work that fails any of these checks is a draft to repair, not a result to use. A piece of work that passes all of them is not necessarily correct—no finite set of checks can guarantee correctness—but it is at least internally consistent in the most common ways.

Appendix B

Pocket Disclosure Template

One page. Print it. Use it.

For each substantive use of an AI system in submitted work, complete the five fields below. Disclosure is required for AI-assisted derivations, code, fits, citations, and any output that contributes to the substance of your work. Disclosure is not required for routine notation lookup, syntax help, or other incidental use.

AI-Use Disclosure

For _____ *(assignment, problem, or section)*,

I used _____ *(AI system, version if known, date)*

in the following way:

1. *What I asked:*

   _____

   _____

   _____

2. *What it returned (in summary):*

   _____

   _____

   _____

3. *What I accepted, in my own words:*

   _____

   _____

   _____

4. *What I rejected, with brief reason:*

   _____

   _____

5. *What I verified independently and how:*

   _____

   _____

   _____

Signature: _____ Date: _____

Appendix C

Further Reading

Primary sources and further reading on the cognitive-science evidence cited in the guide.

The references below are the primary sources for the cognitive-science claims made in Chapter 5. Each is freely available in published form; many are also available as preprints or as open-access editions.

Retrieval practice

Karpicke, J.D., & Roediger, H.L. (2008). The critical importance of retrieval for learning. *Science*, 319(5865), 966–968.

Carpenter, S.K., Pan, S.C., & Butler, A.C. (2022). The science of effective learning with spacing and retrieval practice. *Nature Reviews Psychology*, 1(9), 496–511.

Productive failure

Kapur, M. (2014). Productive failure in learning math. *Cognitive Science*, 38(5), 1008–1022.

Loibl, K., Roll, I., & Rummel, N. (2017). Towards a theory of when and how problem solving followed by instruction supports learning. *Educational Psychology Review*, 29(4), 693–715.

Illusion of fluency

Bjork, R.A. (1994). Memory and metamemory considerations in the training of human beings. In J. Metcalfe & A.P. Shimamura (Eds.), *Metacognition: Knowing About Knowing* (pp. 185–205). MIT Press.

Kornell, N., & Bjork, R.A. (2008). Learning concepts and categories: Is spacing the "enemy of induction"? *Psychological Science*, 19(6), 585–592.

Cognitive offloading

Risko, E.F., & Gilbert, S.J. (2016). Cognitive offloading. *Trends in Cognitive Sciences*, 20(9), 676–688.

Adjacent reading

The following books and articles develop themes adjacent to this guide. They are not required reading but reward attention.

Brown, P.C., Roediger, H.L., & McDaniel, M.A. (2014). *Make It Stick: The Science of Successful Learning.* Harvard University Press. A book-length treatment of retrieval practice and related ideas, written for a general audience.

Lang, J.M. (2016). *Small Teaching: Everyday Lessons from the Science of Learning.* Jossey-Bass. Practical applications of cognitive-science findings to classroom teaching, useful to instructors as well as motivated students.

Willingham, D.T. (2009). *Why Don't Students Like School?* Jossey-Bass. A short book on the cognitive science of learning, accessible and concrete.

Mollick, E. (2024). *Co-Intelligence: Living and Working with AI.* Portfolio. A general-audience account of working with AI tools, with chapters relevant to study and research.

The companion repository for this guide hosts additional resources: prompt-pattern libraries, failure-mode catalogs with worked examples, and an evolving set of community-contributed observations on AI use in technical study. The repository is at https://github.com/machyman/learning-with-ai-student-guide.

Appendix D

Sample Course Syllabus Statements

Three syllabus templates for different course types. Adapt to your course; do not adopt verbatim.

The samples below are starting points for the AI-policy section of a course syllabus. Each is calibrated to a different course type and AI-use posture. Adapt to your course's specific context, your institutional policies, and the assessment plan you have designed (Chapters 10–11).

Sample 1: Foundational graduate course

AI Tool Use Policy. This course covers foundational material that students must master independently of AI tools. AI use is permitted on take-home assignments under the following conditions:

- In-class examinations are conducted without AI tools or network access.
- Take-home problem sets permit AI use, with required disclosure (see Appendix B of *Learning with AI*). The disclosure must include: tools used, prompts, AI responses (in summary), what you accepted and rejected, and what you verified independently.
- Verification is required for any AI-generated derivation, code, or analysis that contributes to your submission. Submitting unverified AI output is not acceptable.
- Course grading: 50% in-class exams (no-AI), 30% take-home work (open-AI with disclosure), 20% oral examinations (no-AI).
- A minimum performance threshold on the no-AI components is required to pass the course.

Students should consult the guide *Learning with AI: A Student and Teacher Guide* (Hyman 2026) for guidance on responsible AI use and disclosure practices.

Sample 2: Research-track graduate seminar

AI Tool Use Policy. This seminar treats AI tools as part of contemporary research practice. Students are expected to use AI tools in much of their work, with the disclosure and verification standards expected of professional researchers.

- All written work must include a complete tool-use disclosure following the five-field template (see Appendix B of *Learning with AI*).
- Process artifacts are graded alongside content. Each submission should include a plan written before tool use, a tool log, a verification checklist, and the final work.
- Two-layer grading is applied: (i) mathematical correctness and exposition, (ii) process integrity and verification. Both must be satisfactory.
- Oral defense is required for all major submissions. If you cannot defend the work in conversation, the work does not count as yours for purposes of the seminar.

- Grading: 30% in-class problems (bounded-AI), 35% take-home problem sets (open-AI), 20% research-style performance task, 15% oral examination.

Sample 3: Advanced undergraduate course

AI Tool Use Policy. AI tools are widely available, and this course addresses how to use them responsibly. The course distinguishes between AI use that supports learning and AI use that bypasses it; the assessments are designed to develop the first habit and discourage the second.

- In-class examinations are conducted without AI. They are the primary evidence of your individual mathematical understanding.
- Some homework problems are designated **[no-AI]** and must be completed without AI assistance. Other problems are designated **[AI-permitted]** and may be completed with AI assistance, with required disclosure.
- For [AI-permitted] problems, your submission must include a brief disclosure: what you asked, what you accepted, and what you verified.
- All students should read *Learning with AI: A Student and Teacher Guide* (Hyman 2026) Part I in the first two weeks of the semester. The reading covers the verification, disclosure, and self-assessment habits this course expects.
- Grading: 60% in-class exams (no-AI), 25% homework (mixed regimes with disclosure), 15% project (open-AI with disclosure).

Notes on adaptation

The samples above are starting points. Three principles when adapting:

Be specific. "You may use AI" is not a policy; it is the absence of one. State explicitly which AI uses are permitted, which are required to be disclosed, which are forbidden, and what the consequences of each are.

Be consistent. The policy on Assignment 7 should not differ from the policy on Assignment 3 unless there is a reason students can understand. Variations across assignments are fine when they reflect different assessment goals; arbitrary variations are corrosive.

Be enforceable. A policy that cannot be enforced is worse than no policy, because it teaches students that policies are optional. If you cannot reliably tell whether a student followed your AI policy, the policy needs to be redesigned, not the students disciplined.

Appendix E

Oral Exam Question Bank

A working set of question types for oral examinations in technical courses.

Oral examinations are the most informative assessment instrument available to instructors in the AI era (Chapter 11). They probe live understanding, they cannot be outsourced, and they provide direct evidence of competencies (D1–D4 and D7) that other instruments measure indirectly. This appendix offers a working bank of question types that can be adapted to specific courses.

The bank is organized into five question categories, each addressing different competencies and progressing roughly from concrete to abstract. A typical 30–60 minute oral exam draws three or four questions across at least three categories.

E.1 Definitions and motivating examples

These questions establish that the student has internalized the basic vocabulary of the course and can connect definitions to motivating examples.

Sample questions:

- State the definition of [core concept] in your own words. Why is the definition formulated this way and not in some weaker or stronger form?
- Give an example of a [type of object] that satisfies [property A] but not [property B]. Why does your example work?
- What is the relationship between [concept X] and [concept Y]? Are they equivalent? Are they independent? Does one imply the other?
- How would you explain [concept] to a strong undergraduate who has not seen it before? What examples would you use? What pitfalls would you warn them about?

These questions assess D1 (conceptual understanding and definitions) and D4 (mathematical communication). A student who genuinely understands a concept can usually answer all four for that concept; a student who has memorized definitions but not internalized them typically fails the second and fourth.

E.2 Short proofs

Five-to-ten-minute arguments built from the standard tools of the course. The questions test whether the student can construct an argument under live conditions, not merely recognize one.

Sample questions:

- Prove [theorem] from [hypotheses]. You may use any standard results from the course, but state them clearly when invoked.
- Here is a claim: [claim]. Is it true? If yes, sketch a proof. If no, give a counterexample.
- State [theorem] and explain its proof at a high level. What is the key insight? What goes wrong if [hypothesis] is removed?

- Here is a half-completed proof of [result]. Complete it, justifying each step.

These questions assess D2 (proof construction) and D3 (technique selection). The fourth variant is particularly informative because it forces the student to engage with someone else's reasoning—which is the same skill needed to evaluate AI-generated proofs (D7).

E.3 Hypothesis stress tests

Modify the assumptions of a known result and ask the student to predict what changes. These questions probe whether the student understands why hypotheses are needed, not just that they are.

Sample questions:

Theorem requires [hypothesis]. What happens to the conclusion if the hypothesis is weakened to [weaker version]? If it is removed entirely?

- Give an example where the hypothesis [X] holds but [Y] does not. What does this tell us about the role of [Y] in [theorem]?
- Here are two theorems that look similar: [T1] and [T2]. What makes them different? Why does the difference in hypotheses matter?
- A student claims that [theorem] still holds when [hypothesis] is replaced by [alternative]. Are they right? If yes, can you sketch why? If no, what counterexample shows the modification fails?

These questions assess D1 (conceptual understanding—specifically the dependency-tracing aspect) and D3 (technique selection). They are among the hardest for AI tools to handle reliably, because they require holding multiple hypothetical scenarios in mind and reasoning across them; a student who answers them well has demonstrated something AI assistance cannot easily provide.

E.4 Audit-and-repair

The student is given a short argument (possibly tool-generated) containing subtle flaws and must locate and repair them. This is the most directly future-proof question type because it operates on the same task that AI literacy demands: detecting errors in AI output.

Sample questions:

- Here is a proof of [claim]. Identify the first invalid step, explain why it is invalid, and describe how to repair the proof (or whether the proof can be repaired at all).
- Here is a derivation of [formula]. The final answer is correct, but the derivation contains an error somewhere. Find the error.
- Here is a piece of code that purports to compute [quantity]. The code runs without error and produces output, but the output is wrong on some inputs. Identify the conditions under which the code fails and explain why.
- Here is a calculation that uses [theorem]. The calculation is correct in form, but [theorem]'s hypotheses are not satisfied for this problem. Identify which hypothesis fails and propose a different approach that does apply.

These questions assess D2 (proof repair), D7 (AI tool proficiency, in the form of error detection), and D8 (research integrity, in the willingness to identify problems rather than gloss over them). They produce strong evidence about student capability and are difficult to prepare for in a way that bypasses understanding.

E.5 Synthesis questions

Connect two or more results from the course in a new way. Synthesis questions are the most cognitively demanding and the most informative about deep understanding.

Sample questions:

- How is [theorem A] related to [theorem B]? Can one be derived from the other? Do they answer different versions of the same underlying question?
- We covered [technique 1] in week 4 and [technique 2] in week 8. Both can be applied to [problem class]. When is each preferable? Are there problems where only one applies?
- Topic A and [topic B] both involve [common concept]. How does the concept appear differently in each? What does this tell us about the concept itself?
- Suppose you were teaching this course. Which two results would you treat as the most important, and why? How would you organize the rest of the material around them?

These questions assess D1, D3, and D4 simultaneously, often with elements of D5 (modeling and interpretation) when relevant. The fourth variant doubles as a question about the student's own development as a mathematician—it elicits their judgment about the field, not just their command of its content.

E.6 Question selection and pacing

Three guidelines for selecting questions and pacing an oral exam.

Mix categories. A 30-minute oral might include one definitions-and-examples question (5 minutes), one short proof (10 minutes), one hypothesis stress test (8 minutes), and one synthesis question (7 minutes). Pure-proof oral exams test a narrow band of competencies; mixed orals produce more useful evidence.

Start accessible, end probing. Open with a definitions or short-proof question that lets the student settle in. Move toward harder questions as the exam progresses. Ending with a synthesis question gives strong students an opportunity to demonstrate range; for weaker students, the difficulty curve allows you to assess where their understanding holds.

Probe with follow-ups. Most of the diagnostic value of an oral exam comes from follow-up questions: "Why?" "What if [variation]?" "Can you say more about [step]?" A student who gives correct one-line answers but cannot extend them under questioning has different competence than one who can develop the argument. The follow-ups reveal which is which.